Cahier de TP
« Gestion centralisée de la sécurité avec KeyCloak»

Pré-requis :

- Bonne connexion Internet
- NodeJS : Version 22+
- npm 10 ou plus (fourni avec Node.js 22)
- Python 3.10 ou plus
- Docker / JDK21
- Git
- IDE Java
- Maven ou Quarkus CLI

Images Docker utilisées :

- postgres:15
- quay.io/keycloak/keycloak:26.8.0
- quay.io/keycloak/keycloak:26.7.4 (Quarkus Dev Services)
- dpage/pgadmin4:9
- axllent/mailpit
- osixia/openldap:1.5.0
- osixia/phpldapadmin:0.9.0
- nginx:1.29
- prom/prometheus:v3.5.0
- grafana/grafana:12.1.1

Les chemins indiqués dans les énoncés de TP sont relatifs à la racine du dépôt git

Certaines solutions sont disponibles à :
https://github.com/dthibau/keycloak-solutions

Table des matières

Atelier 1: Installation et sécurisation d'une première application....4
1.1 Installation....4
1.2. Création d'un realm....4
1.3. Sécurisation d'une application....5
1.3.1 Démarrage des applications....5
1.3.2 Enregistrement du client *frontend*....5
1.3.3 Attribut personnalisé et accès au backend....6
Atelier 2: OpenID Connect....8
2.1 Endpoint de découverte....8
2.2 Authentification....8
2.2.1. Requête d'authentification....8
2.2.2 Obtention des jetons....9
2.2.3 Rafraîchissement....9
2.2.4 UserInfo....10

2.3 User Profile et personnalisation des revendications....10
2.4 Back-channel Logout....10
Atelier 3 : Jetons d'accès OAuth....11
3.1 Consentement de l'utilisateur....11
3.2 Limitation des audiences du jeton....11
3.3 Limitation des rôles du jeton....12
3.4 Limitation des scopes du jeton....12
3.5 Client scopes personnalisés....13
3.6 Validation du jeton via le endpoint d'introspection....13
Atelier 4. Sécurisation d'applications....15
4.1 Applications natives....15
4.1.1 loopback URI....15
4.1.2 Device Code Flow....15
4.2 Client Credentials Flow....16
4.2.1 Client credentials avec secret....16
Atelier 5. Intégration....17
5.1 Intégration javascript....17
5.2 Intégration Spring, application web traditionnelle....17
5.2.1 Logout....19
5.3 Intégration Quarkus....19
5.3.1 Propagation du jeton....20
5.4 Intégration Python....21
5.4.1 Application web Flask....21
5.4.2 API FastAPI....23
Atelier 6 : Stratégies d'autorisation....25
6.1. GBAC....25
6.2. Autorisation services....25
Atelier 7 : Exploitation courante....29
7.1 Gestion des utilisateurs....29
7.2 Mots de passe et récupération de compte....29
7.3 Authentification à deux facteurs....30
7.4 Sessions et jetons....31
7.5 Événements et audit....31
Atelier 8 : Adaptation et personnalisation....32
8.1 Administrateur dédié à un realm....32
8.1.1 Administrateur du realm....32
8.1.2 Permissions fines....32
8.2 Organizations....32
8.2.1 Activation et création....33
8.2.2 Membres et jeton....33
8.3 Intégration à un serveur LDAP....33
8.3.1 Configuration de la fédération....34
8.3.2 Stratégies de fédération....34
8.4 Intégration à un fournisseur d'identité OpenID Connect....35
8.4.1 Fournisseur externe....35
8.4.2 Déclaration dans le realm formation....35
8.4.3 Test....35
8.4.4 Fournisseur rattaché à une organisation....36
8.5 Flow d'authentification....36
8.5.1 Flow MFA : WebAuthn ou OTP....36
8.5.2 Usage du claim acr....37
Atelier 9 : Développement serveur....38

9.1 Thème....38
9.2 Event Listener SPI....38
Option A : développer le listener....38
Option B : avec Docker....39
Activation et test (options A et B)....39
9.3 Admin REST API....39
Atelier 10 : Architecture et fonctionnement....41
10.1 Options de démarrage....41
10.2 Base de données, TLS, reverse proxy et endpoints....43
10.3 Image optimisée....45
Atelier 11 : Haute disponibilité et clustering....47
11.1 Cluster à deux nœuds....47
11.2 Bascule et persistance des sessions....48
Atelier 12 : Configuration avancée et durcissement....49
12.1 Calibrage des sessions et du pool....49
12.2 Gestion des secrets....51
12.3 Défenses de sécurité du realm....52
Atelier 13 : Exploitation et maintenance d’un cluster....54
13.1 Ajout et retrait d’un nœud....54
13.2 Montée de version....54
13.3 Diagnostic d’une dégradation....55
Atelier 14 : Supervision....57
14.1 Prometheus et Grafana....57
14.2 Détecter une saturation ou un problème de réplication....58
Ateliers avancés....59
A1 Authentification utilisateur avec certificat X509 (Optionnel)....59
A2 CIBA Flow (Optionnel)....61
A3 DPoP – Validation d’un appel API protégé....63
A4 Preuve de possession avec mTLS....63
A5 : Client policies....64

Atelier 1: Installation et sécurisation d'une première application

1.1 Installation

JDK21

Page de téléchargement : ***https://www.keycloak.org/downloads***

L'extraire dans un répertoire de votre choix

Démarrer en mode développement

Linux, Git bash :

```
bin/kc.sh start-dev
```

Windows CMD, PowerShell

```
bin/kc.bat start-dev
```

Accéder à http://localhost:8080 et créer un utilisateur admin : ***admin/admin***

Docker

Utiliser le docker compose fourni

```
cd 1_FirstAppli
docker compose up -d
```

Cela démarre 3 services :

- Serveur Keycloak
- Base Postgres
- Client pgAdmin

1.2. Création d'un realm

Accéder à la console d'administration

Créer un realm nommé ***formation***

Pour le realm *formation* :

- Créer un user avec comme login ***user***
- Dans l'onglet ***Credentials,*** définir le mot de passe ***secret*** et décocher la check-box ***temporaire***
- Créer un rôle REALM : ***ROLE_USER***

- Créer un groupe ***GROUP_USER*** et y affecter l'utilisateur précédent, lui affecter également le rôle

Se connecter avec *user/secret* à la console des comptes utilisateur (*http://localhost:8080/realms/formation/account*), et parcourir l'interface.

1.3. Sécurisation d'une application

Répertoire racine : ***TPS/1_FirstAppli***

1.3.1 Démarrage des applications

Démarrer l'application frontend :

```
cd frontend
npm install
npm start
```

Puis dans une autre fenêtre, l'application back-end

```
cd backend
npm install
npm start
```

Accéder à *http://localhost:8000* et vérifier la présence d'un bouton login

1.3.2 Enregistrement du client frontend

Dans la console d'admin de Keycloak, créer un nouveau client :

- Client Type : ***OpenID***
- Client ID: ***frontend***
- Client authentication : ***Off***
- Authorization : ***Off***
- Authentication Flow : ***Standard Flow (Authorization Code)***
- Décocher : ***Direct Access Grant (Password Grant Type)***

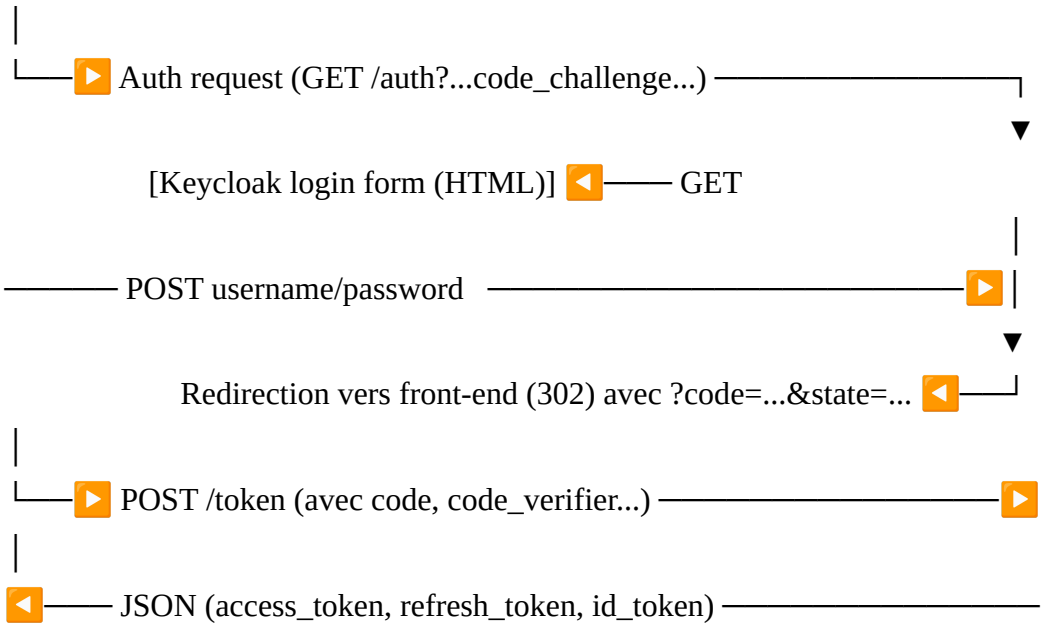
URLs

- Root URL: ***http://localhost:8000***
- Valid Redirect URI : ***http://localhost:8000/***
- Logout URL : + (cela inclut toutes les *Valid Redirect URIs*)
- Web origins : +

Activer ensuite le bouton ***login*** dans l'application *frontend*, et loggez vous avec le login user.

Vous êtes rentrer dans l'application et pouvez accéder aux boutons permettant de voir les jetons obtenus.
Vous pouvez vous délogger et recommencer le processus de login en activant la console de debug du navigateur sur l'onglet réseau et ne filtrer que les requêtes Document et XHR.

Essayer de visualiser les URLs suivantes :
[Frontend SPA – login]



Visualiser les jetons fournis par Keycloak, vérifier que l'utilisateur a bien le rôle ***ROLE_USER***

Access Token.realm_access.roles

1.3.3 Attribut personnalisé et accès au backend

Avec le profil administrateur sur le realm formation, accèder à *Realm Settings – User Profile*

Créer un attribut ***picture***

Indiquer les permissions afin que l'utilisateur et l'admin puisse renseigner l'attribut.

Avec le profil user, renseigner l'attribut ***picture*** avec une URL pointant sur une image afin de voir comment une application peut utiliser les attributs stockés dans *KeyCloak*

Accéder directement aux URLs du backend :

- *http://localhost:3000/public*
- *http://localhost:3000/secured*

Visualisez les traces dans le backend

Accéder ensuite à l’URL sécurisé via l’application de *frontend.*

Visualiser les traces de l’application backend

Atelier 2: OpenID Connect

Répertoire racine : ***TPS/2_OpenIDConnect/authcode_flow***

2.1 Endpoint de découverte

Démarrer l'application :

```
npm install
npm start
```

Accéder à http://localhost:8000

Activer le bouton « *1 – Discovery* »

Vérifier l'URL de l'issuer (il contient le nom du realm Keycloak)

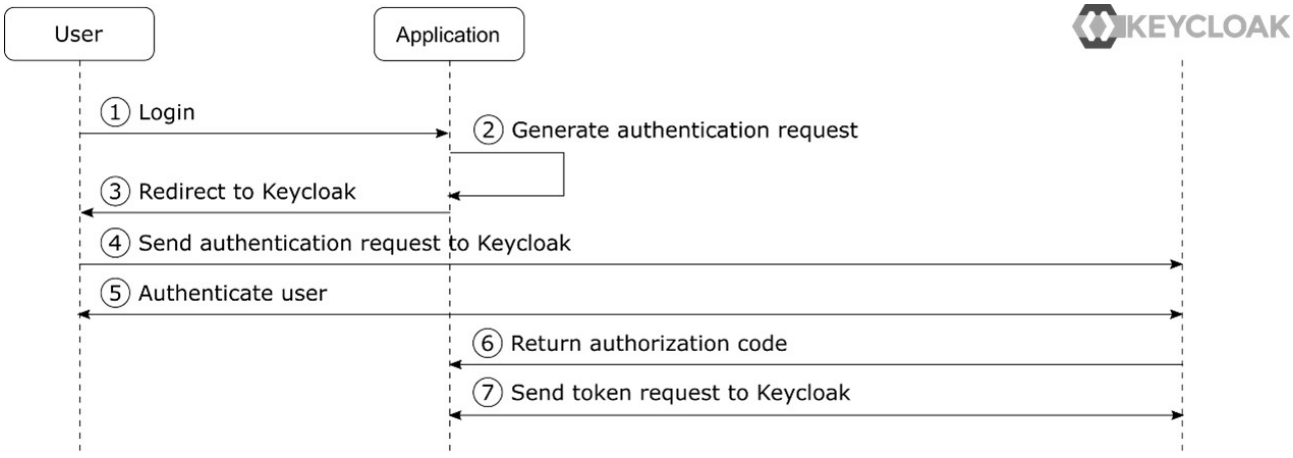
Activer ensuite le bouton « *Load OpenID Provider Configuration* »

Elle provoque la requête GET :
http://localhost:8080/realms/<realm-name>/.well-known/openid-configuration

Visualiser les informations accessibles.

2.2 Authentification

Le processus détaillé de l'authentification est le suivant :



2.2.1. Requête d'authentification

Pour envoyer la requête d'authentification, cliquer sur le bouton correspondant.

Un formulaire est proposé par l'application :

- *client_id* : le client enregistré dans Keycloak

- *scope* : doit contenir *openid* pour une requête OpenID Connect ; d'autres scopes peuvent être ajoutés
- *prompt* : conditionne l'affichage de l'écran de login :
  - *none* : pas d'écran de login, l'utilisateur n'est authentifié que s'il a déjà une session
  - *login* : l'utilisateur doit s'authentifier même s'il a déjà une session
- *max_age* : âge maximal, en secondes, de la dernière authentification
- *login_hint* : identifiant de l'utilisateur, pour pré-remplir le champ login de Keycloak

Générer la requête et l'envoyer. Le point d'accès est :

```
GET http://localhost:8080/realms/<realm-name>/protocol/openid-connect/auth
```

Observer la section Paramètres de sécurité et retrouver state, nonce et code_challenge dans la requête d'authentification générée

La réponse doit contenir le code d'autorisation, accompagné du *state* envoyé. L'application vérifie que le state reçu est identique

2.2.2 Obtention des jetons

La 3ème étape consiste à échanger le code d'autorisation contre les jetons. Utiliser le bouton correspondant.

Observer la réponse et ses différents champs : *access_token*, *expires_in*, *refresh_token*, *refresh_expires_in*, *id_token*, *token_type*, *scope*.

Observer le jeton d'identification décodé et repérer les claims *iss*, *sub*, *aud*, *azp*, *exp*, *iat*, *auth_time* et *nonce*.

Observer le *code_verifier* dans la requête de jeton, puis le *nonce* dans le jeton d'identification déchiffré

2.2.3 Rafraîchissement

Rafraîchir les jetons via l'application, ou avec curl en reprenant le refresh token obtenu :

```
curl -d grant_type=refresh_token -d client_id=<client_id> \
-d refresh_token=$REFRESH_TOKEN \
http://localhost:8080/realms/<realm-name>/protocol/openid-connect/token
```

Comparer les nouveaux jetons aux précédents (*iat*, *exp*).

Réutiliser l'ancien refresh token : il est toujours accepté.

Dans *Realm settings → Tokens*, activer *Revoke Refresh Token* puis recommencer : l'ancien refresh token est désormais refusé. Désactiver ensuite l'option.

2.2.4 UserInfo

Appeler le point d'accès UserInfo avec le jeton d'accès, via l'application ou avec curl :

```
curl -H "Authorization: Bearer $ACCESS_TOKEN" \
http://localhost:8080/realms/<realm-name>/protocol/openid-connect/userinfo
```

Comparer la réponse au contenu du jeton d'identification : elle ne contient pas les claims techniques (*iss*, *aud*, *nonce*…).

2.3 User Profile et personnalisation des revendications

Dans *Realm settings → User profile*, créer un attribut ***no-secu***, avec les permissions de lecture et d'écriture pour l'utilisateur et l'admin (comme l'attribut *picture* de l'atelier 1).

Renseigner l'attribut *no-secu* de l'utilisateur *user*.

Sélectionner le client *frontend*, *Client scopes,→ frontend-dedicated*. Ajouter un mapper *By configuration* de type *User Attribute* :

- Name : *no-secu-mapper*
- User Attribute : *no-secu*
- Token Claim Name : *no-secu*
- Claim JSON Type : *String*
- Add to ID token et Add to userinfo : *On*

S'authentifier à nouveau et vérifier la présence du claim *no-secu* dans le jeton d'identification et dans la réponse UserInfo.

Toujours dans *frontend-dedicated*, ajouter un mapper de type *Group Membership* (Token Claim Name : *groups*) et vérifier la présence du groupe *GROUP_USER* dans les jetons.

2.4 Back-channel Logout

Pour le client frontend, désactiver le *front-channel* logout

Configurer une URI de Back-channel logout à : http://localhost:8000/logout ou http://host.docker.internal:8000/logout

Se logger sur l'application ***frontend***

Avec un autre navigateur, connecter vous en tant qu'admin sur le serveur Keycloak.

Accéder à *Realm Formation → sessions*

Forcer un logout SSO via l'administration

Observer les traces de Keycloak ainsi que celles de l'application *frontend*

Atelier 3 : Jetons d’accès OAuth

Répertoire racine : ***TPS/3_oAuth2***

Démarrer l’application front-end :

```
cd frontend
npm install
npm start
```

Puis l’application back-end

```
cd backend
npm install
npm start
```

Accéder à *localhost:8000*, s’authentifier puis invoquer le service backend.

3.1 Consentement de l’utilisateur

Avec Keycloak, les applications peuvent être configurées pour exiger ou ne pas exiger le consentement.

Dans la console d’administration, sélectionner le client *frontend* et activer le consentement de l’utilisateur.

Obtenir à nouveau un jeton d’accès via *localhost:8000*

Créer un nouveau client scope avec les informations suivantes :

- Name: ***albums***
- Display On Consent Screen: ***ON***
- Consent Screen Text: ***Visualiser votre album photo***

Configurer le client *frontend* pour que ce *scope* soit optionnel.

Obtenir à nouveau un jeton d’accès en précisant le scope albums.

3.2 Limitation des audiences du jeton

Éditer le fichier de configuration du backend *keycloak.json* et passer l’attribut ***verify-token-audience*** à ***true***

Le nom de la ressource est ***backend***

Obtenir à nouveau un jeton d’accès et observer le champ ***aud***

Essayer d’invoquer le service

Dans la console d'admin, créer un client ***backend*** en décochant tous les flows de login

Revenir sur le client *frontend* puis le client scope *frontend-dedicated* et créer un nouveau Mapper de type **audience**:

- Name: ***backend audience***
- Mapper Type: ***Audience***
- Included Client Audience: ***backend***

Essayer d'invoquer le service

3.3 Limitation des rôles du jeton

Rôle realm

Visualiser le jeton d'accès avec la configuration par défaut. Des informations sur les rôles des utilisateurs et les rôles des clients doivent apparaître

Éditer le scope dédié à l'application frontend, accéder à l'onglet *scope* et désactiver « ***Full scope allowed*** »

Visualiser le jeton obtenu avec cette configuration

Retourner dans la configuration du scope dédié et lui affecter le rôle *ROLE_USER*

Visualiser le jeton obtenu avec cette configuration

Retourner dans la configuration du scope dédié et supprimer le rôle *ROLE_USER*

Affecter le rôle USER au client scope *realm_scope* créé au TP précédent

Demander un nouveau jeton en indiquant le scope ***custom_scope***

Rôle client

Ajouter également le rôle ***admin*** au client frontend

Editer le user user et lui affecter le rôle ***front-end:admin***

Visualiser le jeton obtenu

3.4 Limitation des scopes du jeton

Créer 3 scopes :

- ***albums:view***
- ***albums:create***
- ***albums:delete***

Configurer ensuite le client *frontend* afin :

- *album:view* soit un scope par ***défaut***
- *albums:create* et *albums:delete* sont des scopes ***optionnels***

S’assurer que le consentement de l’utilisateur est requis pour le client *frontend*

Demander un nouveau jeton sans indiquer de scope. Le consentement pour le scope *album:view* devrait être demandé

Demander ensuite un nouveau jeton en indiquant un scope optionnel.

3.5 Client scopes personnalisés

Dans le client *frontend → Client scopes → frontend-dedicated*, supprimer le mapper *no-secu-mapper* créé à l’atelier 2.3.

Créer un client scope *secu_scope* (type *None*) et, dans son onglet *Mappers*, un mapper *By configuration* de type *User Attribute* :

- Name : *no-secu-mapper*
- User Attribute : *no-secu*
- Token Claim Name : *no-secu*
- Claim JSON Type : *String*
- Add to ID token : *On*

Ajouter *secu_scope* comme scope optionnel du client *frontend*.

Avec l’application de l’atelier 2, s’authentifier sans demander ce scope : le claim *no-secu* est absent. S’authentifier à nouveau avec le paramètre *scope* : *openid secu_scope* : le claim est présent.

Dans *secu_scope*, onglet *Scope*, affecter le rôle *ROLE_USER*.

Créer un utilisateur *user_with_no_roles* sans le rôle *ROLE_USER* et renseigner son attribut *no-secu*. S’authentifier avec cet utilisateur en demandant *secu_scope* : le claim *no-secu* n’apparaît pas.

Retrouver ces résultats avec la fonction *Client scopes → Evaluate* du client *frontend*.

3.6 Validation du jeton via le endpoint d’introspection

S’assurer que le client ***backend*** a la case cochée « *Client authentication* ».
Récupérer le secret du client backend.

Dans un terminal :

```
export SECRET=….
```

Récupérer le jeton d’accès de l’application front-end, puis

```
export TOKEN=…
```

Valider le jeton via :

```
curl --data "client_id=backend&client_secret=$SECRET&token=$TOKEN"
http://localhost:8080/realms/formation/protocol/openid-connect/tok
en/introspect
```

Atelier 4. Sécurisation d’applications

4.1 Applications natives

4.1.1 loopback URI

Obtenir un jeton pour une application native, sans navigateur embarqué ni secret, en déléguant le login au navigateur du poste. L'application démarre un serveur HTTP local sur un port aléatoire qui sert d'URL de redirection, technique dite « loopback » de la RFC 8252.

Enregistrer un client :

- Client ID: ***cli***
- Access Type: ***public***
- Standard Flow Enabled: ***ON***
- Valid Redirect URIs: ***http://localhost/callback***

Récupérer les sources fournis, les visualiser et les comprendre :

```
npm install
npm start
```

Le démarrage de l’application ouvre la page d’authentification de Keycloak, l’application doit afficher :

- Son port d’écoute
- Le code d’autorisation
- Le jeton obtenu

Après authentification, Keycloak envoie le code d’autorisation à l’appli qui récupère le jeton

4.1.2 Device Code Flow

Modifier le client ***cli*** afin qu’il supporte le grant type ***Device***

Envoyer une requête POST sur le *device_authorization_endpoint*

```
curl -X POST \
    -d "client_id=cli" \
    "http://localhost:8080/realms/formation/protocol/openid-connect/auth/device"
```

Cela retourne une réponse comme suit :


```
{
"device_code": "MABcN2jmFFn8kvy-7w2idCnF4lZcTtmzkFbB_Nz4kHU",
"user_code": "WEWK-HKCH",
"verification_uri": "http://localhost:8080/realms/formation/device",
```

"verification_uri_complete": "http://localhost:8080/realms/formation/device?
user_code=WEWK-HKCH",
"expires_in": 600,
"interval": 5
}

Ouvrir le navigateur à ***verification_uri***. Enter le ***user_code***, s'authentifier et consentir.

Finalement, obtenir le jeton via :

```
curl -X POST \
    -d "grant_type=urn:ietf:params:oauth:grant-type:device_code" \
    -d "client_id=cli" \
   -d "device_code=<device_code>" \
   "http://localhost:8080/realms/formation/protocol/openid-connect/token"
```

4.2 Client Credentials Flow

4.2.1 Client credentials avec secret

Créer un nouveau client :

- Client ID: ***backend***
- Client Protocol: ***openid-connect***
- Client authentication: ***ON***
- Standard Flow Enabled: ***OFF***
- Implicit Flow Enabled: ***OFF***
- Direct Access Grants Enabled: ***OFF***
- Service Accounts Enabled: ***ON***

Obtenir les jetons avec la commande curl suivante :

```
curl --data
"client_id=service&client_secret=$SECRET&grant_type=client_credent
ials" http://localhost:8080/realms/formation/protocol/openid-
connect/token
```

Atelier 5. Intégration

5.1 Intégration javascript

Répertoire racine : ***TPs/5_Integration/javascript/***

Solution disponible

Utiliser le client *frontend* (client public, Standard flow).

Récupérer le code fourni : *npm install* installe le paquet *keycloak-js*, que l'application expose sous */keycloak-js*.

Compléter ensuite l'initialisation de keycloak-js :

- Instancier l'objet *Keycloak* avec *url*, *realm* et *clientId*
  (ou à partir du fichier *keycloak.json* : *Clients → frontend → Action → Download adapter config*, fichier à placer à côté de *index.html* et chargé par *new Keycloak('keycloak.json')*)
- Appeler *init()* avec *onLoad: "login-required"* : la page de login Keycloak s'affiche si l'utilisateur n'est pas authentifié
- Vérifier que PKCE est actif : *pkceMethod: "S256"* (valeur par défaut de keycloak-js 26, l'option le rend explicite)

Avant chaque appel au backend, appeler *updateToken(30)*, puis placer le jeton d'accès dans l'en-tête *Authorization*.

L'application backend fournie est une application Node.js (Express, Node.js 20 ou plus) qui valide elle-même le jeton d'accès avec la bibliothèque *jose* : récupération des clés publiques du realm sur le endpoint JWKS, puis vérification de la signature, de l'issuer, de l'expiration et de l'audience *backend*.

L'adaptateur *keycloak-connect* est déprécié et n'est plus publié depuis janvier 2025, il n'est donc plus utilisé (voir le support).

- Récupérer le code fourni et lire la fonction *requireToken* : issuer attendu, URL du JWKS, audience exigée
- Démarrer le backend : *npm install* puis *npm start*
- Accéder directement à *http://localhost:9000/protected* : réponse 401 et trace « pas de jeton » côté backend

Faire en sorte que le frontend puisse invoquer le backend : Web origins du client *frontend*, et mapper Audience *backend* sur ce client puisque le backend exige *aud = backend*.

En cas de refus, lire la raison dans les traces du backend.

5.2 Intégration Spring, application web traditionnelle

Répertoire racine : ***TPs/5_Integration/springbackend/***

Projet fourni complet, aucun code à écrire

Objectif : Exécuter une application web traditionnelle Spring MVC et observer comment Spring Security délègue l'authentification à Keycloak : login OpenID Connect (authorization code avec PKCE), déconnexion RP-initiated et back-channel logout.

Le projet (Spring Boot 4, Java 17 ou plus) utilise les starters :

- *spring-boot-starter-webmvc*
- *spring-boot-starter-security*
- *spring-boot-starter-oauth2-client*.

La configuration de sécurité s'appuie sur *HttpSecurity* et *SecurityFilterChain*, comme dans le support.

Ouvrir le projet et repérer :

- ***src/main/resources/application.yml*** *:*
  le provider keycloak n'indique que issuer-uri (les endpoints sont lus dans la discovery) ;
  la registration springapp porte le client-id, le secret, le redirect-uri {baseUrl}/login/oauth2/code/keycloak et le scope openid
- ***SpringbackendApplication***, chaîne de sécurité : oauth2Login (login OpenID Connect), oidcLogout (back-channel logout, endpoint /logout/connect/back-channel/springapp), logout avec OidcClientInitiatedLogoutSuccessHandler (redirection vers le end session endpoint de Keycloak, retour sur {baseUrl}/)
- *les autorisations :*
  / public, /oidc-principal réservé à l'autorité OIDC_USER, le reste authentifié
- les endpoints **/oidc-principal** (utilisateur connecté : claims du jeton d'identification et autorités Spring) et **/access-token** (jeton d'accès conservé par Spring)

Déclarer le client *springapp* dans Keycloak avec :

- Client authentication : On
- Standard flow : On
- Root URL et Home URL : http://localhost:8001
- Valid redirect URIs : http://localhost:8001/login/oauth2/code/keycloak
- Valid post logout redirect URIs : http://localhost:8001/*
- Web origins : +
- Back-channel logout URL : http://localhost:8001/logout/connect/back-channel/springapp
  (remplacer localhost par host.docker.internal si Keycloak tourne dans Docker, voir 2.4)

Récupérer le client secret (*Clients → springapp → Credentials*) et démarrer l'application en le passant en argument, ou en le reportant dans *application.yml* :

```
./mvnw spring-boot:run -Dspring-boot.run.arguments=--
spring.security.oauth2.client.registration.springapp.client-
secret=<secret>
```

Accéder à *http://localhost:8001/oidc-principal* et se connecter avec *user* / *secret*.

Dans l'onglet réseau, suivre l'enchaînement : */oauth2/authorization/springapp* (Spring construit la requête avec state, nonce et code_challenge), page de login Keycloak, retour sur */login/oauth2/code/keycloak?code=...*, puis la réponse JSON : claims du jeton d'identification et autorités *OIDC_USER*, *SCOPE_openid*…

Si Spring affiche « invalid_client », le secret n'est pas le bon : Keycloak a authentifié l'utilisateur mais refuse l'échange du code.

Accéder à *http://localhost:8001/access-token* : le jeton d'accès a été obtenu et conservé par Spring lors de l'échange du code, l'application peut le propager vers une API.

5.2.1 Logout

Déconnexion RP-initiated : accéder à *http://localhost:8001/logout* et observer dans l'onglet réseau la redirection vers l'*end_session_endpoint* de Keycloak avec *id_token_hint* et *post_logout_redirect_uri*, puis le retour sur l'application.

Une nouvelle connexion redemande le mot de passe : la session Spring et la session SSO ont toutes deux été fermées.

Back-channel logout : Se reconnecter et noter le claim *sid* affiché par */oidc-principal*.

Dans la console d'administration, menu Users, utilisateur *user*, onglet Sessions : retrouver la session dont l'identifiant correspond à ce *sid* et cliquer « Sign out », ou « Sign out all sessions ».

Keycloak appelle l'URL de back-channel logout, Spring trace « Invalidated session », et le rechargement de l'application redemande une authentification.

Attention à ne pas utiliser la console des comptes utilisateur (*/realms/formation/account*) pour ce test : dans le même navigateur, elle partage la session SSO avec l'application, et son bouton « Sign out all devices » conserve justement la session courante. L'application serait alors reconnectée sans mot de passe au rechargement.

Pour suivre ces échanges, activer les événements de login du realm (Realm settings → Events → User events settings) et consulter le menu Events : LOGIN, LOGOUT et CODE_TO_TOKEN apparaissent avec le client et l'identifiant de session.

5.3 Intégration Quarkus

Répertoire racine : ***TPs/5_Integration/quarkus/***

Projet fourni complet, aucun code à écrire

Objectif : Exécuter deux services Quarkus et observer la sécurisation d'une API par rôles (*delivery-service*) puis la propagation du jeton d'accès vers une seconde API (*notification-service*).

Cet atelier utilise les Dev Services de Quarkus (projets en Quarkus 3.40, Java 17 ou plus) : Docker doit être démarré, et Quarkus lance lui-même un conteneur Keycloak sur un port aléatoire, avec un realm *quarkus*, un client *quarkus-app* et les utilisateurs *alice* (rôles admin et user) et *bob* (rôle user).

Le serveur Keycloak utilisé jusqu'à maintenant peut rester démarré, il n'est pas utilisé ici.

Ouvrir le projet *delivery-service* et repérer :

- dans ***pom.xml***, les extensions *quarkus-oidc* (validation des jetons) et *quarkus-rest-client-oidc-token-propagation* (propagation du jeton vers les appels REST sortants)

- dans ***application.properties***, l'absence de *quarkus.oidc.auth-server-url* : c'est ce qui déclenche les Dev Services ; et *quarkus.rest-client.notification-api.url*, l'adresse de notification-service
- ***org.formation.web.LivraisonController*** : les contrôles d'accès par *@RolesAllowed* (admin sur la classe, user sur les lectures)
- ***org.formation.service.NotificationService*** : le client REST (*@RegisterRestClient*) annoté *@AccessToken*, qui recopie le jeton d'accès reçu dans l'en-tête Authorization des appels sortants
- ***org.formation.service.impl.LivraisonServiceImpl***, méthode *create* : l'appel à *notificationService.sendMail* après la création d'une livraison

Démarrer delivery-service en mode développement :

```
./mvnw quarkus:dev
```

Le premier démarrage télécharge l'image Keycloak (environ 480 Mo).

Si le démarrage échoue avec « Timed out waiting for log output », relancer la commande ou augmenter le délai : *./mvnw quarkus:dev -Dquarkus.devservices.timeout=300s*.

Vérifier le démarrage du conteneur Keycloak et noter le port publié vers 8080 :

```
docker ps
```

Accéder à la Dev UI (*http://localhost:8000/q/dev-ui/*), puis à la carte OpenID Connect :

- Accéder à l'administration Keycloak via le lien proposé, avec *admin/admin*
- Visualiser la configuration du client *quarkus-app* et ses client scopes
- Depuis la Dev UI, se connecter avec *bob/bob* et observer le jeton d'accès : les rôles sont portés par le claim *groups*, que quarkus-oidc utilise pour *@RolesAllowed*
- Appeler une URL en GET (par exemple via Swagger UI, */q/swagger-ui/*), puis essayer un POST : 403, bob n'a pas le rôle admin
- Observer avec F12 le jeton envoyé dans les requêtes

5.3.1 Propagation du jeton

Ouvrir le projet *notification-service* : il dépend aussi de *quarkus-oidc* et ses endpoints sont protégés par le rôle *admin* (*NotificationResource*).

Le démarrer dans un second terminal, il écoute sur le port 9000 :

```
./mvnw quarkus:dev
```

Il ne démarre pas un second Keycloak : les Dev Services partagent le conteneur déjà lancé par *delivery-service* (option *quarkus.keycloak.devservices.shared*, active par défaut), les deux services valident donc les jetons du même realm.

Depuis la Dev UI de delivery-service, se connecter avec *alice/alice* et créer une livraison (POST) : on obtient 201, et la console de notification-service affiche « Sending email » (le mailer est simulé en mode dev).

Le jeton d'alice, qui porte le rôle admin, a été propagé tel quel par *@AccessToken* et accepté par notification-service.

Pour constater le rôle de la propagation (optionnel) : mettre en commentaire l'annotation *@AccessToken* dans *NotificationService*, le mode dev recompile à la requête suivante. Créer une

livraison avec alice : on obtient une erreur 500 dont le détail indique « Unauthorized, status code 401 », l'appel sortant est parti sans jeton. Rétablir l'annotation.

5.4 Intégration Python

Répertoire racine : ***TPs/5_Integration/python/***

Projet fourni complet, aucun code à écrire

Objectif : Exécuter une application web Flask (Authlib) et une API FastAPI (PyJWT) et observer les mêmes mécanismes qu'avec Spring et Quarkus : login OpenID Connect avec PKCE, appel d'une API avec le jeton d'accès, contrôle de l'audience, déconnexion.

Prérequis : Python 3.10 ou plus, et le serveur Keycloak de la formation démarré sur le port 8080 (realm *formation*, utilisateur *user*). Dans le répertoire du projet, créer un environnement virtuel et installer les librairies :

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

5.4.1 Application web Flask

Dans le realm *formation*, créer un client *flask-app* :

- Client authentication : *On*, Standard flow : *On*
- Valid redirect URIs : *http://localhost:5000/auth*
- Valid post logout redirect URIs : *http://localhost:5000/*

Ouvrir *app.py* et repérer :

- ***oauth.register*** : le client est décrit par *server_metadata_url* (discovery), le scope *openid profile email* et *code_challenge_method=S256* (PKCE) ; le secret est lu dans la variable d'environnement *KC_CLIENT_SECRET*
- ***/login*** : *authorize_redirect* construit la requête d'autorisation (state, nonce, code_challenge) et redirige vers Keycloak
- ***/auth*** : *authorize_access_token* vérifie le state, échange le code avec le secret et le code_verifier, valide le jeton d'identification et son nonce ; l'utilisateur et les jetons sont conservés dans la session
- ***/api*** : appel de l'API avec l'en-tête *Authorization: Bearer* ; */token* : affichage des claims du jeton d'accès
- ***/logout*** : session effacée puis redirection vers l'*end_session_endpoint* avec *id_token_hint* (RP-initiated logout)

Version abrégée du fichier fourni, pour référence (le fichier complet ajoute la lecture du secret dans l'environnement, la page */token* et des commentaires) :

```
from flask import Flask, redirect, session, url_for
from authlib.integrations.flask_client import OAuth
import requests
```

```
app = Flask(__name__)
app.secret_key = "change-me"
oauth = OAuth(app)
oauth.register("keycloak",
  client_id="flask-app", client_secret="<secret>",

server_metadata_url="http://localhost:8080/realms/formation/.well-known/openid-configuration",
  client_kwargs={"scope": "openid profile email", "code_challenge_method": "S256"})


@app.route("/")
def index():
  user = session.get("user")
  if not user:
    return '<a href="/login">Se connecter</a>'
  return f'Bonjour {user["preferred_username"]} - <a href="/api">API</a> - <a href="/logout">Logout</a>'


@app.route("/login")
def login():
  return oauth.keycloak.authorize_redirect(url_for("auth", _external=True))


@app.route("/auth")
def auth():
  token = oauth.keycloak.authorize_access_token()
  session["user"] = token["userinfo"]
  session["access_token"] = token["access_token"]
  session["id_token"] = token["id_token"]
  return redirect("/")


@app.route("/api")
def api():
  r = requests.get("http://localhost:8002/api/hello",
    headers={"Authorization": "Bearer " + session["access_token"]})
  return f"{r.status_code} : {r.text}"
```

```
@app.route("/logout")
def logout():
  id_token = session.get("id_token")
  session.clear()
  end = oauth.keycloak.load_server_metadata()
["end_session_endpoint"]
  return redirect(f"{end}?
id_token_hint={id_token}&post_logout_redirect_uri=http://
localhost:5000/")
```

Récupérer le client secret (Clients → flask-app → Credentials) et démarrer l'application :

```
export KC_CLIENT_SECRET=<secret>
flask --app app run --port 5000
```

Accéder à *http://localhost:5000* et se connecter avec *user* / *secret*, puis repérer dans l'onglet réseau les paramètres *state*, *nonce*, *code_challenge* et *code_challenge_method=S256* de la requête d'autorisation : Authlib les génère et les vérifie au retour sur */auth*. Afficher les claims du jeton d'accès via le lien Jeton.

Tester le logout : une nouvelle connexion doit redemander le mot de passe.

5.4.2 API FastAPI

Ouvrir *api.py* et repérer :

- ***PyJWKClient*** : récupération des clés publiques du realm sur le endpoint JWKS, mises en cache
- ***jwt.decode*** : vérification de la signature (RS256), de l'expiration, de l'issuer et de l'audience *python-api*
- ***HTTPBearer*** : extraction du jeton de l'en-tête Authorization ; toute erreur de validation renvoie 401 avec le motif

Version abrégée du fichier fourni, pour référence :

```
import jwt
from fastapi import Depends, FastAPI, HTTPException
from fastapi.security import HTTPBearer

ISSUER = "http://localhost:8080/realms/formation"
jwks = jwt.PyJWKClient(ISSUER + "/protocol/openid-connect/certs")
app = FastAPI()

def current_user(cred = Depends(HTTPBearer())):
  try:
    key = jwks.get_signing_key_from_jwt(cred.credentials)
```

```
    return jwt.decode(cred.credentials, key.key,
algorithms=["RS256"],
                      audience="python-api", issuer=ISSUER)
  except jwt.PyJWTError as e:
    raise HTTPException(status_code=401, detail=str(e))


@app.get("/api/hello")
def hello(claims = Depends(current_user)):
  return {"user": claims["preferred_username"]}
```

Démarrer l'API dans un second terminal, avec le même environnement virtuel :

```
uvicorn api:app --port 8002
```

Depuis l'application Flask, cliquer sur le lien *API* : on obtient *401 : {"detail":"Audience doesn't match"}*, l'audience *python-api* étant absente du jeton (le claim *aud* ne contient que *account*).

Dans le client *flask-app → Client scopes → flask-app-dedicated*, ajouter un mapper By configuration de type *Audience*, avec *Included Custom Audience* : *python-api* et *Add to access token* : On.

Se déconnecter puis se reconnecter pour obtenir un nouveau jeton, et rappeler l'API : on obtient 200 avec le nom de l'utilisateur, le claim *aud* contient maintenant *python-api* et *account*.

Atelier 6 : Stratégies d’autorisation

6.1. GBAC

Choisir le client *backend*.

Ajouter un nouveau mapper avec les informations suivantes :

- Name: ***groups***
- Mapper Type: ***Group Membership***
- Token Claim Name: ***groups***

Créer ensuite un groupe ***« Human Resource »***puis un sous-groupe ***« Manager »***

Affecter l’utilisateur au groupe Manager

Retourner dans *backend → Client Scopes → Evaluate*

Sélectionner le user et générer le jeton d’accès

Observer le claim *groups*, puis désactiver l’option *Full group path* du mapper et comparer.

6.2. Autorisation services

Récupérer l’application *springboot* fournie

L’application (Spring Boot 4) nécessite un JDK 21. Avant de la démarrer, lire les fichiers suivants :

1. *pom.xml* : *spring-boot-starter-oauth2-resource-server* (validation du jeton JWT) et *keycloak-policy-enforcer* (le PEP, Policy Enforcement Point)
2. *src/main/resources/application.yml* : port 8180 et *issuer-uri* du realm
3. *src/main/resources/policy-enforcer.json* : configuration du PEP : realm, URL de Keycloak, client et son secret
4. *SecurityConfiguration.java* : un jeton JWT valide est exigé (sinon 401), puis le filtre policy enforcer interroge Keycloak (sinon 403)
5. *ApplicationController.java* : les endpoints */api/...*, sans aucune règle d’autorisation dans le code

Dans cette application, il existe 2 chemins protégés par des autorisations spécifiques dans Keycloak :

- ****/api/{resource}**** : l'accès à cette ressource est basé sur l'évaluation des permissions associées à une ressource « *Default Resource* » dans Keycloak. tout utilisateur avec un rôle *user* est autorisé à accéder à cette ressource.
- ****/api/premium**** : l'accès à cette ressource est basé sur l'évaluation des permissions associées à une ressource « *Premium Resource* » dans Keycloak. Seuls les utilisateurs avec un rôle *user-premium* sont autorisés à accéder à cette ressource.

Vous pouvez utiliser deux utilisateurs distincts pour accéder à cette application :

- alice/alice : rôle *user*
- jdoe/jdoe : rôles *user* et *user-premium*

La première étape consiste à créer un realm avec les utilisateurs, importer le fichier ***config/quickstart-realm-simple.json*** qui :

1. Crée le realm ***spring-boot-quickstart***
2. Crée les « realm roles » et les utilisateurs
3. Crée le client confidentiel ***app-authz-rest-springboot*** avec l'activation du service d'autorisation
4. Définit les 2 ressources « Default Resource » et « Premium Resource »

Ensuite, via la console d'admin, dans l'onglet *Authorization* du client *app-authz-rest-springboot* :

Créer 2 policies basées sur les rôles :

- Only User Policy : rôle *user*
- Only Premium Policy : rôle *user-premium*

Créer 2 permissions « resource based » associées au policies précédentes

- **Default Resource Permission**
- **Premium Resource Permission**

Démarrer l'application springboot :

```
./mvnw spring-boot:run
```

Obtenir un jeton d'accès, par exemple pour *alice* :

Remarque : le password grant est utilisé ici uniquement pour obtenir un jeton en ligne de commande ; il est à proscrire en production.

Vérifier que *Direct access grants* est activé sur le client.

```
export access_token=$(\
    curl -X POST
http://localhost:8080/realms/spring-boot-quickstart/protocol/openid-connect/
token \
    -H 'Authorization: Basic YXBwLWF1dGh6LXJlc3Qtc3ByaW5nYm9vdDpzZWNyZXQ=' \
    -H 'content-type: application/x-www-form-urlencoded' \
    -d 'username=alice&password=alice&grant_type=password' | jq --raw-output
'.access_token' \
    )
```

Accéder aux ressources :

```
curl http://localhost:8180/api/resourcea \
  -H "Authorization: Bearer "$access_token
```

```
curl http://localhost:8180/api/premium \
  -H "Authorization: Bearer "$access_token
```

On doit obtenir la réponse ***« Access granted »*** ou un ***403*** si la ressource n'est pas permise

Appeler aussi une ressource sans en-tête *Authorization* : réponse 401, Spring Security exige un jeton. Avec un jeton valide, c'est le policy enforcer (PEP) intégré à l'application qui décide. Il ne contient aucune règle : il demande la décision à Keycloak, qui joue le rôle de PDP (Policy Decision Point).

Observer les traces DEBUG de l'application pour l'appel avec le jeton d'accès :

1. le PEP retrouve la ressource Keycloak dont l'URI correspond au chemin appelé (*Default Resource* pour */api/resourcea*)
2. « Obtaining authorization for authenticated user » : il envoie au endpoint token de Keycloak le jeton de l'utilisateur, avec le grant *urn:ietf:params:oauth:grant-type:uma-ticket* et le secret du client
3. Keycloak évalue les policies et renvoie un nouveau jeton, le RPT (Requesting Party Token), qui liste les permissions accordées, ou *403 access_denied* si aucune permission ne s'applique
4. « Authorization GRANTED » : le PEP laisse passer la requête jusqu'au contrôleur

Pour voir ce que le PEP a fait, refaire cet échange à la main : demander à Keycloak un RPT à partir du jeton d'accès, pour la permission « Default Resource » :

```
export rpt=$(curl -X POST \

http://localhost:8080/realms/spring-boot-quickstart/protocol/openid-connect/token \
 -H "Authorization: Bearer "$access_token \
 --data "grant_type=urn:ietf:params:oauth:grant-type:uma-ticket" \
 --data "audience=app-authz-rest-springboot" \
  --data "permission=Default Resource" | jq --raw-output '.access_token' \
 )
```

Visualiser le jeton JWT (RPT) et son claim *authorization.permissions*

Le RPT est un jeton d'accès ordinaire (même issuer, mêmes rôles, audience *app-authz-rest-springboot*) enrichi d'un claim *authorization* qui liste les ressources et scopes auxquels l'utilisateur a droit. C'est la décision de Keycloak, signée par lui.

Accéder aux ressources avec ce jeton

```
 curl http://localhost:8180/api/resourcea \
   -H "Authorization: Bearer ${rpt}"
```

Comparer les traces avec l'appel précédent : avec le RPT, le PEP trouve la permission « Default

Resource » directement dans le jeton et accorde l'accès sans rappeler Keycloak. Appeler ensuite */api/premium* avec ce même RPT :

```
curl http://localhost:8180/api/premium \
-H "Authorization: Bearer ${rpt}"
```

La permission n'est pas dans le jeton : le PEP retente l'échange auprès de Keycloak, qui refuse car alice n'a pas le rôle *user-premium* (403).

Demander enfin un RPT pour la permission « Premium Resource » avec le jeton d'alice : Keycloak répond directement *access_denied / not_authorized*. Refaire les manipulations avec *jdoe*.

Question : quel intérêt une application cliente a-t-elle à demander elle-même un RPT et à l'envoyer à l'API plutôt que son jeton d'accès ?

Atelier 7 : Exploitation courante

Le répertoire racine est ***TPs/7_Exploitation***

7.1 Gestion des utilisateurs

Required actions :
Dans *Authentication → Required actions*, vérifier que *Terms and Conditions* est activée.

Créer un utilisateur *jdupont* avec un email, lui donner un mot de passe temporaire et lui affecter les required actions *Update Password*.

Se connecter avec *jdupont* sur la console du compte (http://localhost:8080/realms/formation/account) et observer l'enchaînement des actions.

Self-registration **:**
Activer *Realm settings → Login → User registration*, puis créer un compte depuis la page de login. Observer que le formulaire reprend les attributs du User Profile.

7.2 Mots de passe et récupération de compte

Démarrer un serveur de mail écoutant sur localhost:8025

```
docker compose -f 7.2_Password/docker-compose-mail.yaml up -d
```

Renseigner un email pour le compte *admin* du realm *master*.

Puis, dans *Realm settings → Email* du realm *formation*, indiquer

- Host : *localhost*,
- Port : 1*025*,
- From : *noreply@formation.local*, et tester la connexion.

Activer *Forgot password* dans *Realm settings → Login*, puis dérouler le parcours « mot de passe oublié » avec *jdupont*.

Lire le mail dans Mailpit : *http://localhost:8025*.

Politique de mots de passe **:**
Dans *Authentication → Policies → Password policy*, ajouter

- *Minimum length* (12),
- *Not username*
- *Not recently used* (3).

Changer le mot de passe de *jdupont* depuis la console du compte et vérifier les refus.

Brute force **:**
Dans *Realm settings → Security defenses → Brute force detection*, choisir *Lockout temporarily* avec 3 échecs au maximum.

Échouer 3 fois avec *jdupont*, constater le blocage, puis débloquer le compte depuis la fiche de l'utilisateur.

7.3 Authentification à deux facteurs

Objectif : ajouter un second facteur (code à usage unique TOTP) à un utilisateur, voir où Keycloak le range et comment le flux de connexion le demande.

Préparation :

- Installer *FreeOTP*, *Google Authenticator* ou *Microsoft Authenticator* sur un mobile.
  Sans smartphone, installer l'outil en ligne de commande *oathtool* (paquet *oathtool*) :
  oathtool --totp -b <clé> calcule le code à partir de la clé affichée par Keycloak.
- Dans *Authentication → Policies → OTP Policy*, lire la politique par défaut : type Time Based, algorithme SHA1, 6 chiffres, période 30 s, Look ahead window 1, « Reusable code » désactivé. Ne rien modifier.

Enrôlement par l'utilisateur :

- Se connecter avec *user* sur la console du compte *http://localhost:8080/realms/formation/account*, puis *Account security → Signing in*.
  Dans la section *Two-factor authentication*, *Authenticator application* est marqué « not set up » : cliquer *Set up Authenticator application*.
- Page « Mobile Authenticator Setup » : scanner le QR code avec l'application mobile, ou cliquer *Unable to scan?* pour afficher la clé et les paramètres (Type, Algorithm, Digits, Interval), à saisir à la main ou à passer à *oathtool*.
- Saisir le code à 6 chiffres et un *Device Name*, laisser *Sign out from other devices* coché, valider.
- De retour dans *Signing in*, l'authenticator apparaît avec son nom.
  Côté administrateur, *Users → user → Credentials* montre maintenant deux credentials : *Password* et *OTP* (type totp, le libellé est le Device Name).
  Show data affiche les paramètres : digits, period, algorithm.

Test de connexion :

- Se déconnecter (*Sign out*) puis se reconnecter : après le mot de passe, Keycloak affiche la page de saisie du code à usage unique (*One-time code*).
- Saisir un code faux : « Invalid authenticator code ». Saisir ensuite le bon code. Remarque : un code déjà utilisé est refusé jusqu'à la fenêtre de 30 s suivante, conséquence de « Reusable code » désactivé.
- Dans le menu *Events*, retrouver le *LOGIN_ERROR invalid_user_credentials* du code faux : pour Keycloak, un OTP incorrect est une erreur de credential comme un mauvais mot de passe.

Simuler la perte du téléphone :

- En tant qu'admin, *Users → user → Credentials*, supprimer le credential *OTP*.
- Se reconnecter avec *user* : aucun code n'est demandé. C'est le sous-flux conditionnel vu en préparation : l'utilisateur n'a plus d'OTP configuré, l'étape est sautée.

- Pour imposer un nouvel enrôlement, dans la fiche de l'utilisateur, onglet *Details*, ajouter *Configure OTP* dans *Required user actions* et enregistrer. À la connexion suivante, la page d'enrôlement s'affiche avant d'accéder à l'application.
- Pour aller plus loin : *Authentication → Required actions → Configure OTP → Set as default action* impose l'enrôlement à tout nouvel utilisateur du realm.

7.4 Sessions et jetons

Se connecter avec *user* sur la console du compte et sur l'application de l'atelier 2.

Dans le menu *Sessions*, visualiser les sessions actives.

Retrouver les mêmes informations dans l'onglet *Sessions* de l'utilisateur et dans celui du client.

Fermer la session de *user* depuis l'administration, puis vérifier que le rafraîchissement du jeton échoue dans l'application de l'atelier 2.

Dans le client *frontend*, onglet *Advanced → Advanced settings*, fixer *Access Token Lifespan* à 1 minute.

Vérifier la valeur de *exp* avec *Client scopes → Evaluate*.

Avec l'application de l'atelier 2, demander un jeton avec le scope *openid offline_access*, puis observer la session de type *Offline* dans l'onglet *Sessions* du client.

7.5 Événements et audit

Dans *Realm settings → Events*, activer *Save events* pour les user events (expiration de 7 jours) et pour les admin events, avec *Include representation*.

Générer des événements :

Un login réussi, un login avec un mauvais mot de passe, la modification d'un utilisateur dans la console.

Consulter le menu *Events* :

Filtrer les *LOGIN_ERROR*, puis observer la représentation JSON d'un admin event.

Retrouver les événements en ligne de commande avec ***kcadm*** :

```
$KEYCLOAK_HOME/bin/kcadm.sh config credentials --server http://localhost:8080 \
  --realm master --user admin --password admin
$KEYCLOAK_HOME/bin/kcadm.sh get events -r formation -q type=LOGIN_ERROR
```

Observer enfin la trace des erreurs de login dans les logs du serveur

Atelier 8 : Adaptation et personnalisation

8.1 Administrateur dédié à un realm

8.1.1 Administrateur du realm

Dans le realm ***formation***, créer un utilisateur ***admin-formation*** et lui définir un mot de passe non temporaire.

Dans l'onglet ***Role mapping*** de l'utilisateur, ***Assign role*** → filtrer sur les rôles de clients, puis affecter le rôle ***realm-admin*** du client ***realm-management***.

Se connecter ensuite avec cet utilisateur à la console d'administration du realm formation :

http://localhost:8080/admin/formation/console

Vérifier que seul le realm ***formation*** est administrable.

8.1.2 Permissions fines

Objectif : un utilisateur ***support-paris*** ne peut gérer que les membres du groupe ***paris***.

- Dans le realm ***formation***, ***Realm settings*** → onglet ***General*** : activer ***Admin Permissions*** et sauvegarder
- Créer deux groupes ***paris*** et ***lyon***, et un utilisateur dans chacun (***user*** dans paris, ***jdupont*** dans lyon)
- Créer l'utilisateur ***support-paris*** et lui affecter uniquement les rôles ***query-users*** et ***query-groups*** du client ***realm-management***
- Menu ***Permissions*** → ***Create permission*** → type ***Groups*** :
  - Name : ***manage-paris***
  - Groupe spécifique : ***paris***
  - Scopes : ***view-members***, ***manage-members***
  - Policy : créer une policy de type User désignant ***support-paris***

Se connecter à la console du realm formation avec ***support-paris*** : vérifier qu'il voit et modifie ***alice*** mais pas ***bob***.

L'onglet ***Evaluation*** du menu Permissions permet de vérifier les droits effectifs d'un utilisateur.

8.2 Organizations

Une organisation regroupe, dans le realm ***formation***, les utilisateurs d'une entreprise cliente ou partenaire : des membres, des domaines de messagerie et, plus loin (8.4.4), un fournisseur d'identité.

Tout reste dans un seul realm : mêmes clients, mêmes flows, même SSO.

Prérequis : Mailpit de l'atelier 7.2 démarré et le serveur SMTP du realm configuré.

8.2.1 Activation et création

- ***Realm settings → General → Organizations*** : On, puis Save. Un menu ***Organizations*** apparaît dans la barre latérale.
- ***Organizations → Create organization*** : Name ***ACME***, Alias ***acme***, Domain ***acme.com***. Après création, onglet ***Attributes*** : ajouter ***country*** = ***FR***.
- Ouvrir une fenêtre de navigation privée sur http://localhost:8080/realms/formation/account : la page de login ne demande plus que l'identifiant ou l'email, le mot de passe vient à l'étape suivante. C'est la connexion « identity-first » : Keycloak détermine d'abord l'organisation de l'utilisateur à partir du domaine de son email.

8.2.2 Membres et jeton

- ***Organizations → ACME → Members → Add member*** : sélectionner ***user***. Son type de membership est ***Unmanaged*** : l'administrateur l'a ajouté à la main, Keycloak ne gère pas son cycle de vie.
- ***Clients → frontend → Client scopes → Evaluate*** avec ***user*** : le jeton ne contient pas de claim ***organization***.
  Dans l'onglet ***Client scopes*** du client, ***Add client scope → organization*** en Default (ce scope est fourni par Keycloak). Regénérer le jeton : il contient ***organization : ["acme"]***.
- Vérifier avec l'application de l'atelier 2 (requête d'authentification puis jetons) ou avec le menu *Evaluate* : le même claim apparaît dans le jeton d'accès.
- ***Members → Invite member*** : email ***carol@acme.com***, prénom et nom.
  Lire le mail dans Mailpit (***http://localhost:8025***) et suivre le lien : Keycloak ouvre le formulaire d'inscription pré-rempli ; une fois le mot de passe choisi, ***carol*** apparaît dans ***Members***.
- Question : quel usage une application peut-elle faire du claim ***organization*** ? Comparer avec le claim ***groups*** de l'atelier 6.1.

8.3 Intégration à un serveur LDAP

Arrêter la stack persistante et démarrer la stack de l'atelier.

Le fichier docker-compose comprend :

- Un serveur Keycloak en mode dev
- Un serveur OpenLDAP qui importe un fichier LDIF au démarrage
- Un client phpLdapAdmin qui permet d'inspecter le serveur LDAP

```
cd 8_Admin/8.2_LDAP
docker compose up -d
```

Vérifier le contenu de l'annuaire en accédant à https://localhost:6443 et se connecter avec :

```
cn=admin,dc=cyberlab,dc=local / admin
```

Repérer les utilisateurs, les groupes LDAP et leurs membres.

8.3.1 Configuration de la fédération

Accéder à Keycloak et créer un realm ***formation***.

Dans ***User federation*** → ***Add LDAP providers***, renseigner le formulaire avec les champs suivants :

- Vendor : ***Other***
- Connection URL : ***ldap://ldap-server:389*** (tester la connexion)
- Bind DN : ***cn=admin,dc=cyberlab,dc=local***
- Bind credentials : ***admin*** (tester l'authentification)
- Edit mode : ***READ_ONLY***
- Users DN : ***ou=users,dc=cyberlab,dc=local***
- Username LDAP attribute : ***cn***
- RDN LDAP attribute : ***cn***
- UUID LDAP attribute : ***entryUUID*** (identifiant immuable fourni par OpenLDAP)
- User object classes : ***inetOrgPerson***

Laisser les autres paramètres à leur valeur par défaut, puis sauvegarder.

Dans l'onglet ***Mappers*** du provider, ajouter un mapper de type ***group-ldap-mapper*** :

- LDAP Groups DN : ***ou=groups,dc=cyberlab,dc=local***
- Group object classes : ***groupOfNames***

Laisser les autres paramètres par défaut.

Depuis le menu ***Action*** du provider, lancer ***Sync all users***, puis depuis le mapper de groupes ***Sync LDAP groups to Keycloak***.

Se connecter à la console account avec un utilisateur LDAP, par exemple ***user1*** / ***password*** : http://localhost:8080/realms/formation/account

Créer un client ***appli*** avec un mapper ***Group Membership*** et vérifier la présence des groupes dans le jeton d'accès (onglet ***Client scopes*** → ***Evaluate***).

8.3.2 Stratégies de fédération

- Ouvrir la fiche de ***user1*** dans ***Users*** : repérer le lien de fédération et les champs non modifiables
- Mode READ_ONLY : tenter de modifier le nom de famille et constater le refus
- Passer l'Edit mode à ***UNSYNCED***, modifier le nom de famille de ***user1*** : vérifier dans phpLdapAdmin que l'annuaire est inchangé
- Passer l'Edit mode à ***WRITABLE***, modifier l'email de ***user2*** : vérifier dans phpLdapAdmin que la modification a été écrite
- Créer un utilisateur ***user-kc*** dans Keycloak : vérifier qu'il est créé dans ***ou=users***

Optionnel : depuis le menu ***Action*** du provider, ***Remove imported*** ; désactiver ***Import users*** (Edit mode READ_ONLY ou WRITABLE) puis se reconnecter avec ***user1***. L'utilisateur est désormais lu à la volée dans l'annuaire.

8.4 Intégration à un fournisseur d'identité OpenID Connect

Objectif : déléguer l'authentification des utilisateurs du realm ***formation*** à un fournisseur d'identité externe.

Dans une intégration réelle, ce fournisseur serait Google, Microsoft Entra ID, FranceConnect ou le Keycloak d'un partenaire.

Pour ne dépendre d'aucun compte externe, l'atelier fait jouer les deux rôles au même serveur : un second realm, ***third-party-provider***, tient lieu de fournisseur externe. Un realm est totalement isolé des autres (utilisateurs, clients, clés de signature, sessions) ; vu du realm ***formation***, il est aussi étranger qu'un serveur distant.

Qui joue quel rôle :

- realm ***third-party-provider*** : le fournisseur d'identité (OpenID Provider). Il possède l'utilisateur ***third-party-user*** et son mot de passe.
- client ***broker-app***: c'est le realm ***formation*** vu par le fournisseur.
  Keycloak s'en servira, pour échanger le code d'autorisation contre les jetons, exactement comme l'application ***frontend*** de l'atelier 1 est cliente du realm ***formation***.
  Sa Valid redirect URI est le endpoint de broker du realm ***formation*** (***/realms/formation/broker/oidc/endpoint***), là où le fournisseur renvoie le code.

Arrêter la stack avec l'annuaire LDAP et reprendre le démarrage du serveur via la ligne de commande

8.4.1 Fournisseur externe

Créer un realm ***third-party-provider***, puis un client avec les paramètres suivants :

- Client ID : ***broker-app***
- Client authentication : ***On***, Standard flow coché
- Valid redirect URIs : ***http://localhost:8080/realms/formation/broker/oidc/endpoint***

Récupérer le secret dans l'onglet ***Credentials***.

Créer un utilisateur ***third-party-user*** avec un mot de passe et un email ***third-party-user@acme.com***, sans prénom ni nom.

8.4.2 Déclaration dans le realm formation

Dans le realm ***formation***, créer un rôle de realm ***partner***.

Dans ***Identity providers***, ajouter un fournisseur ***OpenID Connect v1.0*** :

- Alias : ***oidc***
- Discovery endpoint : http://localhost:8080/realms/third-party-provider/.well-known/openid-configuration
- Client ID : ***broker-app*** et son secret

Dans l'onglet ***Mappers*** du fournisseur, ajouter un mapper ***Hardcoded Role*** qui attribue le rôle ***partner***.

8.4.3 Test

Ouvrir la console account du realm formation : http://localhost:8080/realms/formation/account

- La page de login propose un bouton oidc : l'utiliser et s'authentifier avec third-party-user
- Observer le first broker login : la page Review profile demande les informations manquantes (prénom, nom)
- Dans la console d'administration, vérifier que l'utilisateur a été créé, qu'il a le rôle ***partner*** et consulter l'onglet ***Identity provider links***

Se déconnecter, puis construire une requête d'autorisation sur le client ***appli*** (créé en 8.3) avec le paramètre ***kc_idp_hint*** :

```
http://localhost:8080/realms/formation/protocol/openid-connect/auth?client_id=appli
  &response_type=code&scope=openid&redirect_uri=<redirect URI de appli>&kc_idp_hint=oidc
```

Constater que la page de login du realm formation est court-circuitée.

8.4.4 Fournisseur rattaché à une organisation

Au 8.4.3, le fournisseur ***oidc*** est proposé à tous par un bouton, ou choisi par ***kc_idp_hint***. Rattaché à l'organisation ***ACME*** du 8.2, il est choisi automatiquement d'après le domaine de l'email saisi.

- Pour repartir d'un utilisateur inconnu du realm ***formation***, supprimer ***third-party-user*** dans ***Users*** (il a été créé par le first broker login du 8.4.3).
- ***Organizations → ACME → Identity providers → Link identity provider*** : Identity provider ***oidc***, Domain ***acme.com***, ***Hide on login page*** On, ***Redirect when email domain matches*** On, puis Link.
- Navigation privée sur http://localhost:8080/realms/formation/account : saisir ***third-party-user@acme.com***.
  Keycloak redirige directement vers la page de login du realm ***third-party-provider***, sans bouton ni ***kc_idp_hint***. S'authentifier : retour sur la console du compte après le first broker login.
- ***Organizations → ACME → Members*** : ***third-party-user*** est membre de type ***Managed*** : créé et géré via le fournisseur de l'organisation.
  Avec ***Client scopes → Evaluate*** sur le client ***frontend***, son jeton contient ***organization : ["acme"]***.
- Question : saisir ***dave@other.com*** sur la page de login ; que se passe-t-il ? Quelle différence avec le bouton ***oidc*** et le ***kc_idp_hint*** du 8.4.3 ?

8.5 Flow d'authentification

8.5.1 Flow MFA : WebAuthn ou OTP

Objectif : login par nom d'utilisateur et mot de passe, puis second facteur au choix (WebAuthn ou OTP).

Dans ***Authentication*** → ***Create flow***, créer un flow ***browser-mfa*** (type Basic flow) comme suit :

```
Cookie ALTERNATIVE
Identity Provider Redirector ALTERNATIVE
Formulaires (sous-flux) ALTERNATIVE
   Username Password Form REQUIRED
   Second facteur (sous-flux) REQUIRED
      WebAuthn Authenticator ALTERNATIVE
      OTP Form ALTERNATIVE
```

Avec Chrome, se connecter à la console account avec un utilisateur du realm formation et, dans ***Account security*** → ***Signing in***, enregistrer une Security key (ou une passkey) et une application d'authentification OTP.

Utiliser ce flow pour la console account : ***Clients*** → ***account-console*** → ***Advanced*** → ***Authentication flow overrides*** → Browser flow : ***browser-mfa***.

Se déconnecter puis se reconnecter : tester WebAuthn, puis OTP via le lien ***Try another way***.

Question : que se passe-t-il pour un utilisateur qui n'a configuré aucun second facteur ?

Atelier 9 : Développement serveur

9.1 Thème

Démarrer la stack fournie : elle monte le répertoire ***themes*** dans Keycloak et démarre le serveur sans cache de thèmes.

Créer un répertoire ***mytheme*** et, dans ***mytheme/login***, un fichier ***theme.properties*** contenant :

```
parent=base
import=common/keycloak
```

Se connecter à la console d'administration, créer le realm ***formation*** et un utilisateur, puis dans ***Realm settings*** → ***Themes*** choisir ***mytheme*** comme Login theme.

On applique le thème au realm formation et non à master pour ne pas risquer de bloquer la connexion à la console d'administration.

Accéder à http://localhost:8000/realms/formation/account et visualiser la page de login.

Éditer ***theme.properties***, remplacer ***base*** par ***keycloak*** et actualiser la page de login.

Sous ***mytheme/login***, créer un répertoire ***resources/css*** et un fichier ***styles.css*** :

```
.login-pf body {
    background: DimGrey none;
}
```

Ajouter dans ***theme.properties*** la ligne suivante, puis actualiser :

```
styles=css/styles.css
```

Remplacer ensuite la ligne par la suivante et comparer :

```
styles=css/login.css css/styles.css
```

Remplacer le logo Keycloak par une image : la placer dans ***resources/img*** et la référencer depuis ***styles.css*** (***url(../img/logo.png)***).

9.2 Event Listener SPI

Deux options au choix : développer son propre listener (option A) ou déployer le listener fourni avec Docker (option B).

Option A : développer le listener

Créer un projet Maven avec le ***pom.xml*** fourni.

Implémenter l'interface ***EventListenerProvider***, en particulier ces deux méthodes, en affichant un message sur la console :

- ***public void onEvent(Event event)***
- ***public void onEvent(AdminEvent event, boolean includeRepresentation)***

Implémenter l'interface ***EventListenerProviderFactory***, en particulier :

- ***public EventListenerProvider create(KeycloakSession session)***
- ***public String getId()***

Créer le fichier ***src/main/resources/META-INF/services/org.keycloak.events.EventListenerProviderFactory*** qui référence la classe factory.

Construire le projet :

```
mvn clean package
```

Copier le jar dans le répertoire ***providers/*** de la distribution Keycloak, puis redémarrer le serveur (le build est automatique sans --optimized) :

```
bin/kc.sh start-dev
```

Vérifier la présence du provider dans l'onglet ***Provider info*** de la console (Server info ou Realm info selon la version) : rechercher ***eventsListener***.

Option B : avec Docker

Observer le fichier docker-compose fourni : le volume qui monte le jar dans /opt/keycloak/providers/ et la commande de démarrage. Démarrer la stack.

Activation et test (options A et B)

Dans ***Realm settings*** → ***Events*** → ***Event listeners***, ajouter le listener (***sysout*** pour l'option B, votre identifiant pour l'option A) et sauvegarder.

Effectuer des connexions, déconnexions et opérations d'administration, et surveiller la console du serveur.

9.3 Admin REST API

Récupérer un jeton d'administration :

- Soit avec le grant password et le client ***admin-cli*** du realm master (compte admin)
- Soit avec un client dédié du realm ***formation*** : client confidentiel ***admin-api***, ***Service accounts roles*** activé, puis dans l'onglet ***Service accounts roles*** affecter les rôles ***manage-users***, ***view-realm*** et ***view-clients*** du client ***realm-management***

```
curl -s -d grant_type=client_credentials -d client_id=admin-api -d
client_secret=$SECRET \

http://localhost:8080/realms/formation/protocol/openid-connect/tok
en | jq -r .access_token
```

Effectuer les requêtes suivantes sur le realm ***formation*** :

- Liste des utilisateurs : ***GET /admin/realms/formation/users***
- Création d'un utilisateur : ***POST /admin/realms/formation/users***
- Ajout d'un rôle de realm : ***POST /admin/realms/formation/users/{id}/role-mappings/realm***
- Mappers d'un client : ***GET /admin/realms/formation/clients?clientId=appli*** puis ***GET …/clients/{id}/protocol-mappers/models***

Documentation : https://www.keycloak.org/docs-api/latest/rest-api/index.html

Refaire les mêmes opérations avec ***kcadm.sh*** (***config credentials*** avec ***--client admin-api --secret …***).

Atelier 10 : Architecture et fonctionnement

Les trois ateliers de cette partie utilisent un seul nœud Keycloak : le cluster est abordé à l'atelier 11. Les fichiers se trouvent dans ***10_Production***.

Arrêter auparavant la stack persistante pour libérer les ports 8080, 9000 et 443.

10.1 Options de démarrage

```
cd 10_Production/10.1_Demarrage
```

Mode développement

```
docker run --name kc --rm -p 8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e
KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:26.8.0 start-dev
```

- Repérer dans les logs l'avertissement sur le mode développement, la base utilisée et le temps de démarrage
- Se connecter à http://localhost:8080/admin : comment la console signale-t-elle que le compte admin est temporaire ?

Dans un second terminal, afficher la configuration effective :

```
docker exec kc /opt/keycloak/bin/kc.sh show-config
```

Repérer le profil courant et, pour chaque option, la source de sa valeur. Arrêter le conteneur (Ctrl+C).

Mode production

Démarrer en mode production sans aucune option :

```
docker run --rm quay.io/keycloak/keycloak:26.8.0 start
```

Lire le message d'erreur : quelle option manque-t-il ? Pourquoi le mode production l'exige-t-il ?

Démarrer avec le minimum requis :

```
docker run --name kc --rm -p 8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e
KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:26.8.0 start
--hostname=http://localhost:8080 --http-enabled=true
```

- Comparer avec start-dev : messages au démarrage, durée, avertissement sur la base de données
- Que signifie l'option ***--http-enabled*** ? Dans quel cas l'utilise-t-on en production ?

Arrêter le conteneur.

Fichier de configuration et options de build

Examiner ***conf/keycloak.conf*** : repérer les options d'exécution et les options de build. Démarrer en montant ce fichier :

```
docker run --name kc --rm -p 8080:8080 -p 9000:9000 \
  -v "$PWD/conf/keycloak.conf:/opt/keycloak/conf/keycloak.conf:ro"
\
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e
KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:26.8.0 start
```

Dans un second terminal, lister les fichiers modifiés par le conteneur, puis vérifier que l'image n'a pas changé :

```
docker diff kc | grep lib/quarkus
docker images quay.io/keycloak/keycloak
```

- Les options ***health-enabled*** et ***metrics-enabled*** sont des options de build : repérer dans les logs la reconstruction automatique du serveur
- Interroger l'interface de management :

```
curl -s localhost:9000/health/ready
curl -s localhost:9000/health/live
curl -s localhost:9000/metrics | head -30
```

Vérifier que ***/health*** n'est pas servi sur le port applicatif :

```
curl -s -o /dev/null -w "%{http_code}\n"
localhost:8080/health/ready
```

Précédence des sources

Le fichier fixe ***log-level=info***. Relancer en ajoutant une variable d'environnement, avant le nom de l'image :

```
-e KC_LOG_LEVEL=debug
```

Les logs passent-ils en DEBUG ? Vérifier la source de ***log-level*** avec show-config. Relancer en ajoutant aussi l'option suivante à la fin de la commande :

```
start --log-level=warn
```

Quelle valeur l'emporte ? Classer les trois sources par ordre de précédence.

Fichier de configuration dédié

Examiner ***conf/debug.conf***, puis démarrer avec ce fichier à la place de ***keycloak.conf*** :

```
docker run --name kc --rm -p 8080:8080 -p 9000:9000 \
  -v "$PWD/conf/debug.conf:/opt/keycloak/conf/debug.conf:ro" \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e
KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:26.8.0 start
--config-file=/opt/keycloak/conf/debug.conf
```

Se connecter à la console account (http://localhost:8080/realms/master/account) et observer les traces des événements d'authentification.

Arrêt

Dans un second terminal, arrêter le conteneur proprement et observer les logs d'arrêt :

```
docker stop kc
```

Questions : quelle source de configuration choisir pour les options communes à tous les environnements ? Pour les valeurs propres à un environnement ? Pour les secrets ?

10.2 Base de données, TLS, reverse proxy et endpoints

```
cd ../10.2_Production
```

Étape 1 : base de données

Générer une autorité de certification de formation et le certificat du serveur (utilisés à l'étape 2) :

```
./certs/gen-certs.sh
```

Examiner ***conf/keycloak.conf***, ***.env*** et ***docker-compose.yml***. Comment le mot de passe de la base parvient-il à Keycloak ?

```
docker compose up -d
docker compose logs -f keycloak
```

- Repérer la reconstruction due à db=postgres, la création du schéma (Liquibase) et les adresses d'écoute
- Accéder à pgAdmin (http://localhost:81, admin@formation.local / admin) et enregistrer le serveur PostgreSQL : host ***keycloak-postgresql***, utilisateur et mot de passe ***keycloak***
- Parcourir les tables de la base keycloak, en particulier ***realm***, ***user_entity***, ***databasechangelog*** et ***databasechangeloglock***

Vérifier le lien entre la disponibilité de la base et la readiness de Keycloak :

```
curl --cacert certs/ca.crt -s https://localhost:9000/health/ready
docker compose stop keycloak-postgresql
curl --cacert certs/ca.crt -s https://localhost:9000/health/ready
```

```
docker compose start keycloak-postgresql
```

Étape 2 : TLS

Keycloak expose directement HTTPS sur le port 8443 avec le certificat généré. Ouvrir https://localhost:8443/admin : le navigateur ne reconnaît pas le certificat. Importer ***certs/ca.crt*** comme autorité de confiance dans le navigateur, ou accepter l'exception.

Examiner le certificat présenté (émetteur, noms alternatifs), puis interroger le serveur en ligne de commande :

```
curl -sS https://localhost:8443/realms/master

curl --cacert certs/ca.crt -s
https://localhost:8443/realms/master/.well-known/openid-
configuration \
  | jq "{issuer, authorization_endpoint, token_endpoint}"
```

Pourquoi la première commande échoue-t-elle ? Pourquoi l'interface de management (port 9000) est-elle aussi en HTTPS ?

Étape 3 : reverse proxy

Ajouter Nginx devant Keycloak.

Examiner ***docker-compose.proxy.yml*** et ***nginx/keycloak.conf*** : les variables ***KC_**** du fichier proxy l'emportent sur ***conf/keycloak.conf***.

```
docker compose -f docker-compose.yml -f docker-compose.proxy.yml
up -d
```

- Nginx termine le TLS et appelle Keycloak en HTTP sur le réseau Docker : quelles options rendent cela possible côté Keycloak ?
- Ouvrirhttps://localhost/realms/master/.well-known/openid-configuration : quel est maintenant l'issuer ?
- Ouvrir la console d'administration via l'adresse dédiée : https://admin.localhost/admin

Restreindre les chemins exposés : dans ***nginx/keycloak.conf***, décommenter les deux règles ***location*** du serveur public, puis redémarrer Nginx et tester :

```
docker compose -f docker-compose.yml -f docker-compose.proxy.yml
restart nginx

curl --cacert certs/ca.crt -s -o /dev/null -w "%{http_code}\n"
https://localhost/admin/

curl --cacert certs/ca.crt -s -o /dev/null -w "%{http_code}\n"
https://localhost/realms/master/.well-known/openid-configuration

curl --cacert certs/ca.crt -s -o /dev/null -w "%{http_code}\n" \
  --resolve admin.localhost:443:127.0.0.1
https://admin.localhost/admin/
```

Question : sans l'option proxy-headers, quelle adresse IP apparaîtrait dans les événements de connexion ?

Étape 4 : endpoints et hostname

Une application qui tourne dans le réseau Docker appelle Keycloak directement, sans passer par le proxy. Simuler cet appel :

```
docker compose -f docker-compose.yml -f docker-compose.proxy.yml
--profile tools run --rm -T curl \
  -s http://keycloak:8080/realms/master/.well-known/openid-
configuration \
  | jq "{issuer, authorization_endpoint, token_endpoint,
jwks_uri}"
```

Toutes les URL renvoyées sont publiques : l'application devrait passer par le proxy pour obtenir ses jetons.

Dans ***docker-compose.proxy.yml***, décommenter ***KC_HOSTNAME_BACKCHANNEL_DYNAMIC*** et ***KC_HOSTNAME_DEBUG***, recréer le conteneur Keycloak puis refaire l'appel :

```
docker compose -f docker-compose.yml -f docker-compose.proxy.yml
up -d keycloak
```

- Quelles URL changent ? Pourquoi authorization_endpoint reste-t-il public ?
- Ouvrir https://admin.localhost/realms/master/hostname-debug et comparer les URL frontend, backend et administration

Questions : parmi les ports publiés par cette stack (443, 8443, 9000, 81), lesquels garder en production ? Quel point d'accès les sondes du load balancer doivent-elles utiliser ?

```
docker compose -f docker-compose.yml -f docker-compose.proxy.yml
down
```

10.3 Image optimisée

Cet atelier reprend la configuration de 10.2 dans une image construite à l'avance. Il utilise le reverse proxy et les certificats de 10.2.

```
cd ../10.3_Image
```

Examiner le ***Dockerfile*** et ***conf/keycloak.conf*** :

- Quelles options sont figées dans l'étape de build ? Lesquelles restent des options d'exécution ?
- Pourquoi l'image ne contient-elle ni mot de passe ni certificat ?

Construire l'image :

```
docker compose build
docker image ls keycloak-formation
```

Dans les logs de build, repérer le message indiquant que la configuration est persistée. Démarrer la

stack :

```
docker compose up -d
docker compose logs -f keycloak
```

- Comparer le temps de démarrage avec celui de 10.2 (ligne « started in »). Pourquoi n'y a-t-il plus de reconstruction ?
- Afficher la configuration effective et repérer les options persistées dans l'image :

```
docker compose exec keycloak /opt/keycloak/bin/kc.sh show-config
```

Tester le changement d'une option de build au démarrage :
ajouter ***KC_METRICS_ENABLED: "false"*** à l'environnement du service keycloak, recréer le conteneur, observer les logs puis interroger ***http://localhost:9000/metrics***. La valeur a-t-elle été prise en compte ? Retirer ensuite la variable.

Vérifications

- Accéder à pgAdmin (http://localhost:81) et enregistrer le serveur : host ***keycloak-postgresql***, utilisateur et mot de passe ***keycloak***
- Visualiser les tables de la base keycloak
- Accéder à la console d'administration : https://admin.localhost/admin (admin / admin)

Reprise d'un realm

Exporter le realm formation de l'environnement de développement : ***Realm settings*** → ***Action*** → ***Partial export***, en incluant groupes, rôles et clients.

- Méthode 1 : dans la console de production, ***Create realm*** → ***Resource file***, choisir le fichier exporté
- Méthode 2 : supprimer le realm, copier le fichier dans ***import/***, décommenter la ligne ***command*** du docker-compose (option ***--import-realm***) et recréer le conteneur

Questions : que contient l'export, et que manque-t-il (utilisateurs, secrets) ? Quelle méthode retenir pour initialiser un environnement de production ?

Atelier 11 : Haute disponibilité et clustering

11.1 Cluster à deux nœuds

Arrêter les stacks de l'atelier 10 et la stack persistante.

La stack ***11_Cluster*** reprend la configuration de l'atelier 10 :

- base PostgreSQL ***keycloak-postgresql***,
- Keycloak en HTTP derrière un reverse proxy (ici HAProxy, qui sait sonder les nœuds ; sans TLS pour simplifier les ateliers), accès admin séparé (port 8081), realm importé au démarrage avec ***--import-realm***.
  S'y ajoutent un second nœud
- Mailpit,
- et en option Prometheus et Grafana.

Les options de build sont figées dans le ***Dockerfile*** comme en 10.3.

Les nœuds sont toutefois démarrés avec ***start*** et non ***start --optimized*** : on pourra ainsi changer des options de build pendant les ateliers.

Le cluster démarre volontairement en 26.7.4, pour la montée de version de l'atelier 13.2.

```
cd 11_14_Cluster
docker compose build
```

Examiner le ***Dockerfile***, le fichier ***.env***, ***haproxy/haproxy.cfg*** et le ***docker-compose.yml***.

Démarrer d'abord un seul nœud :

```
docker compose up -d keycloak-postgresql kc1 haproxy mailpit
docker compose logs kc1 | grep ISPN000094
```

La vue du cluster ne compte qu'un membre. Démarrer le second nœud :

```
docker compose up -d kc2
docker compose logs kc1 kc2 | grep ISPN000094
```

Vérifier que chaque nœud voit une vue du cluster à deux membres.

Consulter la table de découverte ***jgroups_ping*** :

```
docker compose exec keycloak-postgresql psql -U keycloak -d
keycloak -c "select * from jgroups_ping;"
```

Ouvrir les statistiques HAProxy : http://localhost:8404 : les deux nœuds doivent être UP.

Demander plusieurs jetons et observer le nœud qui répond (en-tête X-Keycloak-Node) :

```
./scripts/token.sh
./scripts/token.sh
```

```
./scripts/token.sh
```

Dans le navigateur, se connecter à la console account avec ***alice*** / ***alice*** : http://localhost:8080/realms/cluster-lab/account

Dans les outils de développement, repérer le cookie ***KC_ROUTE*** et l'en-tête ***X-Keycloak-Node*** des réponses.

Question : pourquoi les requêtes curl alternent-elles entre les nœuds alors que celles du navigateur restent sur le même nœud ?

Dans la console d'administration (http://localhost:8081/admin), menu ***Sessions*** du realm cluster-lab : la session d'alice est-elle visible quel que soit le nœud qui l'a créée ?

11.2 Bascule et persistance des sessions

Dans un second terminal, lancer une boucle de demandes de jetons, qui reste active pendant tout l'atelier :

```
./scripts/watch.sh
```

Repérer le nœud qui sert le navigateur (cookie ***KC_ROUTE***), puis l'arrêter, par exemple :

```
docker compose stop kc1
```

- Observer la sortie de *watch.sh* : des requêtes ont-elles échoué ?
- Actualiser la console account dans le navigateur : alice est-elle toujours connectée ? Quel nœud répond ?
- Observer la nouvelle vue du cluster dans les logs de kc2

```
docker compose start kc1
```

Arrêter maintenant les deux nœuds, puis les redémarrer :

```
docker compose stop kc1 kc2
docker compose start kc1 kc2
```

Actualiser la console account : alice est-elle toujours connectée ? Consulter les sessions persistées en base :

```
docker compose exec keycloak-postgresql psql -U keycloak -d keycloak \
  -c "select offline_flag, count(*) from offline_user_session group by offline_flag;"
```

Question : comment garantir la persistance des sessions après une bascule ? Quels sont le coût et les limites de cette garantie ?

Atelier 12 : Configuration avancée et durcissement

12.1 Calibrage des sessions et du pool

Profil de charge de la production :

- 20 000 utilisateurs, dont 3 000 se connectent pendant l'heure de pointe
- Journée de travail de 8 h, sans ressaisie du mot de passe souhaitée
- Une application mobile qui doit rester connectée d'un jour à l'autre et rafraîchit son jeton en arrière-plan
- Un cluster de 2 nœuds, auquel un 3e nœud peut être ajouté à chaud, et une base PostgreSQL limitée à 200 connexions

Configuration retenue pour ce profil :

- SSO Session Idle : ***2 h*** (couvre la pause déjeuner sans ressaisie)
- SSO Session Max : ***10 h*** (couvre la journée, reconnexion quotidienne)
- Access Token Lifespan : ***5 min*** (fenêtre d'exposition courte en cas de vol de jeton)
- Application mobile : sessions offline (scope ***offline_access***) et access token de ***10 min*** (moitié moins de rafraîchissements, donc d'écritures en base)
- Offline Session Idle : ***30 jours*** ; Offline Session Max Limited activé, ***90 jours*** (les sessions offline ne s'accumulent pas indéfiniment)
- Pool par nœud : max ***60***, min et initial ***20*** (200 connexions moins 20 réservées, réparties sur 3 nœuds pour pouvoir en ajouter un)

Configuration générale du realm

Dans le realm cluster-lab, ***Realm settings*** → onglet ***Sessions*** :

- SSO Session Idle : ***2 Hours***, SSO Session Max : ***10 Hours***
- Offline Session Idle : ***30 Days***
- Offline Session Max Limited : activé, Offline Session Max : ***90 Days***

Sauvegarder, puis onglet ***Tokens*** : vérifier que Access Token Lifespan vaut ***5 Minutes***.

Pool de connexions

Dans ***docker-compose.yml***, ajouter à l'environnement commun des nœuds (bloc ***x-keycloak***) :

```
KC_DB_POOL_INITIAL_SIZE: "20"
KC_DB_POOL_MIN_SIZE: "20"
KC_DB_POOL_MAX_SIZE: "60"
```

Recréer les nœuds un par un :

```
docker compose up -d --no-deps kc1
docker compose up -d --no-deps kc2
```

Vérifier le pool de kc1 au repos :

```
curl -s localhost:9001/metrics | grep -E "^agroal_(active|
available|max_used)_count"
```

Une vingtaine de connexions sont disponibles (***available***) et aucune n'est utilisée (***active***) : les connexions minimales sont ouvertes dès le démarrage. Compter côté base les connexions ouvertes par les deux nœuds :

```
docker compose exec keycloak-postgresql psql -U keycloak -d
keycloak \
  -c "select count(*) from pg_stat_activity where usename =
'keycloak';"
```

Pourquoi ce total est-il d'environ 40 ? Combien de connexions les nœuds pourraient-ils ouvrir au maximum ? Et avec un 3e nœud ?

Application mobile

Clients → ***Create client*** :

- Client type : ***OpenID Connect***, Client ID : ***mobile-app***
- Client authentication : ***Off*** (une application mobile ne peut pas garder un secret : client public)
- Authentication flow : ***Standard flow*** coché ; ***Direct access grants*** coché uniquement pour obtenir des jetons en ligne de commande dans ce TP
- Valid redirect URIs : ***com.formation.mobile:/oauth2redirect***

Sauvegarder, puis onglet ***Advanced***, section ***Advanced settings*** (lien dans la colonne de droite) :

- Access Token Lifespan : ***10 Minutes*** (surcharge la valeur du realm pour ce client)
- Proof Key for Code Exchange Code Challenge Method : ***S256***

Enregistrer avec le bouton ***Save*** de cette section. Dans l'onglet ***Client scopes***, vérifier que ***offline_access*** est présent en ***Optional***.

Obtenir un jeton offline pour alice et comparer avec le client load-test :

```
curl -s -d grant_type=password -d client_id=mobile-app -d
username=alice -d password=alice \
  -d "scope=openid offline_access" \
  http://localhost:8080/realms/cluster-lab/protocol/openid-
connect/token \
  | jq "{expires_in, refresh_expires_in}"
curl -s -d grant_type=password -d client_id=load-test -d
username=alice -d password=alice \
  http://localhost:8080/realms/cluster-lab/protocol/openid-
connect/token \
  | jq "{expires_in, refresh_expires_in}"
```

- Comparer les durées ***expires_in*** des deux clients
- Décoder le refresh token du client mobile sur jwt.io : quelle est la valeur de ***typ*** ?
- Dans la fiche d'alice, onglet ***Sessions*** : distinguer les sessions en ligne et offline

12.2 Gestion des secrets

Vault de realm

Mailpit est configuré pour exiger une authentification SMTP (***keycloak*** / ***S3cr3t***).

Les nœuds sont démarrés avec ***KC_VAULT=file*** et ***KC_VAULT_DIR=/opt/keycloak/vault***, monté depuis le répertoire ***vault***.

Créer le secret, sans retour à la ligne final :

```
printf 'S3cr3t' > vault/cluster-lab_smtp__password
```

Renseigner une adresse email pour le compte admin (menu utilisateur → ***Manage account***), puis dans le realm cluster-lab, ***Realm settings*** → ***Email*** :

- From : ***noreply@lab.local***
- Host : ***mailpit***, port ***1025***
- Authentication activée, username ***keycloak***, password ***${vault.smtp_password}***

Utiliser ***Test connection*** et vérifier la réception dans Mailpit : http://localhost:8025

Modifier le contenu du fichier de secret, refaire le test. Pourquoi aucun redémarrage n'est-il nécessaire ?

Faire un export partiel du realm : que contient le champ du mot de passe SMTP ?

Keystore de configuration

Le mot de passe de la base est aujourd'hui fourni en variable d'environnement : il est visible par quiconque peut lancer ***docker inspect*** sur un nœud. On le place dans un keystore chiffré, lu par Keycloak au démarrage.

Vérifier d'abord que le mot de passe est visible :

```
docker inspect $(docker compose ps -q kc2) | grep KC_DB_PASSWORD
```

Créer le keystore dans le répertoire ***secrets***, avec une entrée ***kc.db-password***.

Saisir ***keycloak*** (mot de passe de la base) aux deux invites :

```
docker run --rm -it -u "$(id -u):$(id -g)" -v "$PWD/secrets:/s" \
  --entrypoint keytool keycloak-cluster:26.7.4 \
  -importpass -alias kc.db-password -keystore /s/conf.p12 \
  -storepass keystorepass -storetype PKCS12
ls -l secrets/conf.p12
```

Dans ***docker-compose.yml***, bloc commun ***x-keycloak*** :

- Supprimer la ligne ***KC_DB_PASSWORD*** : une variable d'environnement l'emporterait sur le keystore
- Ajouter ***KC_CONFIG_KEYSTORE: /opt/keycloak/secrets/conf.p12*** et ***KC_CONFIG_KEYSTORE_PASSWORD: keystorepass***
- Ajouter le volume ***./secrets:/opt/keycloak/secrets:ro***, dans le bloc commun et dans la liste ***volumes*** de kc1, qui a la sienne

Appliquer la modification à kc2 seul, kc1 continuant de servir les utilisateurs :

```
docker compose up -d --no-deps kc2
docker compose logs -f kc2
```

- kc2 doit démarrer et rejoindre le cluster (***ISPN000094*** avec deux membres) et passer UP dans les statistiques HAProxy
- Vérifier que le mot de passe n'apparaît plus : ***docker inspect $(docker compose ps -q kc2) \| grep KC_DB_PASSWORD***

Une fois kc2 opérationnel, appliquer la même modification à kc1 :

```
docker compose up -d --no-deps kc1
```

12.3 Défenses de sécurité du realm

La restriction des endpoints exposés (/admin, /realms/master) a été mise en place en 10.2, étape 3, au niveau du reverse proxy. Cet atelier porte sur les défenses configurées dans Keycloak lui-même.

Protection contre le brute force

Dans le realm cluster-lab, activer l'enregistrement des événements (***Realm settings*** → ***Events*** → ***User events settings***), puis ***Security defenses*** → ***Brute force detection*** :

- Mode : ***Lockout temporarily***
- Max login failures : ***3***
- Wait increment : ***5 minutes***

Sauvegarder. Provoquer des échecs, puis tenter aussitôt un login avec le bon mot de passe :

```
for i in 1 2 3 4; do ./scripts/token.sh alice mauvais; done
./scripts/token.sh alice alice
```

- Le dernier login échoue-t-il malgré le bon mot de passe ? Les échecs sont-ils comptés alors que les requêtes passent par les deux nœuds ?
- Dans la console d'administration, ***Users*** → alice → onglet ***Details*** : constater le verrouillage (Temporarily locked) et le lever
- Consulter les événements ***LOGIN_ERROR*** dans ***Events***
- Le message renvoyé au client révèle-t-il le verrouillage ?

Le verrouillage n’interrompt pas les sessions déjà ouvertes : il bloque les nouvelles authentifications. Une console account déjà connectée continue donc de fonctionner.

En-têtes de sécurité

Ouvrir la page de login du realm cluster-lab dans une fenêtre privée et relever, dans les outils de développement, les en-têtes de sécurité de la réponse (***Content-Security-Policy***, ***X-Frame-Options***, ***Strict-Transport-Security***…).

Realm settings → ***Security defenses*** → ***Headers*** : passer ***X-Frame-Options*** à ***DENY***, sauvegarder et vérifier l’en-tête renvoyé avec la commande suivante :

```
curl -s -D - -o /dev/null \
  "http://localhost:8080/realms/cluster-lab/protocol/openid-connect/auth?client_id=mobile-app&response_type=code&scope=openid&redirect_uri=com.formation.mobile%3A%2Foauth2redirect&code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM&code_challenge_method=S256" \
  | grep -i -E "^(x-frame-options|content-security-policy)"
```

. Remettre ensuite la valeur par défaut.

Atelier 13 : Exploitation et maintenance d’un cluster

13.1 Ajout et retrait d’un nœud

Garder **./scripts/watch.sh** actif dans un terminal pendant tout l’atelier.

Ajouter un troisième nœud à chaud :

```
docker compose --profile scale up -d kc3
docker compose logs -f kc1 kc3
```

- Observer la nouvelle vue du cluster à trois membres et le rééquilibrage des caches
- Dans les statistiques HAProxy, attendre que kc3 passe UP
- Vérifier dans la fenêtre watch que les trois nœuds répondent

Retirer kc3 proprement (SIGTERM), puis arrêter kc2 brutalement (SIGKILL) :

```
docker compose stop kc3
docker compose kill kc2
```

Comparer les deux cas : erreurs dans watch.sh, délai avant la nouvelle vue du cluster, messages dans les logs de kc1.

Les nœuds sont démarrés avec ***KC_SHUTDOWN_DELAY=5s*** et HAProxy teste les nœuds toutes les 3 s (***inter 3s fall 2***). Ce délai est-il suffisant ? Proposer une valeur.

Réintégrer kc2 et vérifier le retour à un cluster de deux membres :

```
docker compose start kc2
```

13.2 Montée de version

Le cluster tourne en 26.7.4 (fichier ***.env***). Générer les métadonnées de la version en cours, avec la même configuration :

```
chmod a+w upgrade
docker compose run --rm --no-deps kc2 update-compatibility metadata --file=/upgrade/meta-old.json
ls -l upgrade/
```

Sauvegarder la base :

```
docker compose exec keycloak-postgresql pg_dump -Fc -U keycloak keycloak > upgrade/kc-before.dump
```

Passer ***KC_VERSION=26.8.0*** dans ***.env***, construire la nouvelle image, puis vérifier la

compatibilité :

```
docker compose build kc1
docker compose run --rm --no-deps kc2 update-compatibility check --file=/upgrade/meta-old.json
echo $?
```

Interpréter le code retour. Le changement de version mineure impose ici un recreate :

```
docker compose stop kc1 kc2
docker compose up -d kc1
docker compose logs -f kc1
```

Observer la migration du schéma, contrôler /health/ready et un login, puis démarrer kc2 :

```
docker compose up -d kc2
```

Mesurer l'interruption dans watch.sh.

Rolling update d'un changement de configuration

Générer de nouvelles métadonnées (version 26.8.0) :

```
docker compose run --rm --no-deps kc2 update-compatibility metadata --file=/upgrade/meta-old.json
```

Modifier la configuration commune, par exemple ***KC_LOG_LEVEL: "INFO,org.keycloak.events:debug"***, puis refaire le check : le code retour doit valoir 0.

Remplacer les nœuds un par un, en attendant que chacun soit UP dans HAProxy avant de passer au suivant :

```
docker compose up -d --no-deps kc1
docker compose up -d --no-deps kc2
```

Vérifier dans watch.sh qu'aucune requête n'a échoué.

Si une version corrective 26.8.x est disponible, refaire l'exercice en changeant uniquement le numéro de version : le check doit autoriser le rolling update.

Question : quelle serait la procédure de retour arrière après la montée en 26.8.0 ?

13.3 Diagnostic d'une dégradation

Scénario : le lundi à 8 h 30, après le redémarrage des nœuds pendant la maintenance de la nuit, les utilisateurs arrivent tous en même temps. Les caches sont vides : chaque login lit l'utilisateur, ses credentials et ses rôles en base, puis écrit sa session.

Préparation du jeu de données

La production compte des milliers d'utilisateurs ; le TP n'en a que deux. Créer 1 000 utilisateurs, une seule fois (quelques minutes, chaque mot de passe est haché) :

```
./scripts/create-users.sh 1000
```

Pic de connexions après un redémarrage

Recréer les nœuds un par un, en attendant que chacun soit UP dans HAProxy : comme après une maintenance, leurs caches sont vides.

```
docker compose up -d --no-deps kc1
docker compose up -d --no-deps kc2
```

Dans un second terminal, suivre en continu le pool de kc1, puis lancer le pic de connexions :

```
./scripts/pool.sh 9001
./scripts/login-storm.sh 1000 40
```

- Observer la montée de ***active*** et de ***max_used*** pendant le test, et vérifier que ***awaiting*** reste à 0
- Relever les temps de réponse (moyen, médian, p95, max) et les codes HTTP

Même pic avec des caches chauds

Relancer immédiatement le test, sans redémarrer les nœuds : les utilisateurs sont maintenant dans le cache.

```
./scripts/login-storm.sh 1000 40
```

- Comparer ***max_used*** et les temps de réponse avec le test précédent
- Pourquoi ce second test sollicite-t-il beaucoup moins la base ?
- Pourquoi le premier pic après un redémarrage est-il le moment le plus exigeant pour la base ?
- Quelle métrique surveiller pour détecter une saturation du pool avant les utilisateurs ? Avec quel seuil d'alerte ?

Atelier 14 : Supervision

14.1 Prometheus et Grafana

Télécharger les dashboards officiels de Keycloak, puis démarrer Prometheus et Grafana :

```
./monitoring/download-dashboards.sh
docker compose --profile monitoring up -d prometheus grafana
```

Collecte des métriques

Dans Prometheus (http://localhost:9090), menu ***Status*** → ***Target health*** :

- kc1, kc2 et HAProxy doivent être UP ; kc3 est DOWN, ce qui est normal tant qu'il n'est pas démarré
- Examiner ***monitoring/prometheus.yml*** : pourquoi ajoute-t-on les labels namespace, container et pod ?

Dashboards Grafana

Générer d'abord de l'activité : redémarrer les nœuds pour vider les caches, puis lancer un pic de connexions.

```
docker compose restart kc1 kc2
./scripts/login-storm.sh 1000 40
```

Ouvrir Grafana (http://localhost:3000, admin / admin), dossier Keycloak. Régler la période sur les 15 dernières minutes et le rafraîchissement sur 10 s. Dans le dashboard ***Keycloak troubleshooting dashboard***, déplier les sections une à une et repérer :

- ***SLO Metrics*** : disponibilité, pourcentage de réponses en moins de 250 ms, réponses en erreur
- ***Database Metrics*** : utilisation du pool et threads en attente de connexion (lien avec le 13.3)
- ***HTTP Metrics*** : requêtes actives, requêtes par URI et par résultat
- ***Cluster Metrics*** : taille du cluster et temps de réponse entre nœuds
- ***JVM Metrics*** : mémoire et pauses GC ; le panneau KUBERNETES - CPU Usage reste vide, pourquoi ?

Relancer ***./scripts/login-storm.sh 1000 40*** sans redémarrer les nœuds, puis comparer les panneaux SLO et Database avec le premier pic.

Ouvrir ensuite le ***Keycloak capacity planning dashboard*** : suivre ***Password validations rate*** et ***Login Events Rate*** pendant un nouveau pic.

Provoquer des erreurs

Générer des échecs de connexion, puis observer le panneau Error responses et la requête sur les événements :

```
for i in $(seq 1 30); do ./scripts/token.sh bob mauvais >
/dev/null; done
```

14.2 Détecter une saturation ou un problème de réplication

Saturation du cache des sessions

Limiter le cache des sessions avec ***KC_CACHE_EMBEDDED_SESSIONS_MAX_COUNT: "200"***, redémarrer les nœuds, puis générer de nombreuses sessions :

```
./scripts/load.sh 2000 20
```

Dans Prometheus, suivre :

```
vendor_statistics_approximate_entries{cache="sessions"}
rate(vendor_statistics_evictions{cache="sessions"}[1m])
```

Les sessions évincées sont-elles perdues ? Quel est l'effet sur la base de données ? Retirer ensuite l'option.

Partition du cluster

Repérer le réseau Docker de la stack et isoler kc2 :

```
docker network ls
docker network disconnect 11_cluster_default $(docker compose ps -q kc2)
```

- Observer ***vendor_cluster_size*** sur kc1 et l'état des cibles dans Prometheus
- Observer les logs de kc1 et l'état de kc2 dans HAProxy

Reconnecter kc2 en lui rendant son nom sur le réseau, puis observer la fusion des vues (ISPN000093) :

```
docker network connect --alias kc2 11_cluster_default $(docker compose ps -q kc2)
```

Ateliers avancés

A1 Authentification utilisateur avec certificat X509 (Optionnel)

Revisitez le docker compose fourni, en particulier :

- La configuration du certificat serveur dans un keystore
- L'activation de l'authenticator X509
- Les options de la JVM donnant accès à un truststore
- L'option HTTPS_CLIENT_AUTH=request
- Les traces pour l'authentification

A1.1 Création de l'utilisateur alice

Dans le realm formation, créer un utilisateur ***alice*** sans définir de mot de passe

A1.2 Modification du Browser-flow

Dans le menu ***Authentication***, sélectionner le flow ***browser*** et dupliquer le dans un flow nommé ***browser X509***

Ajouter une étape alternative ***X509/Validate Username Form*** au même niveau que ***Cookie***

Configurer là comme suit :

- Alias : ***Use CN***
- User Identity Source : ***Match subject DN using regular expression***
- Regular expression : ***CN=(.*?)(?:,|$)***
- User Mapping Method : ***Username or email***

Dans la liste déroulante Action, sélectionner ***Bind Flow*** et associer ce flow au ***Browser Flow***

A1.3 Test de la configuration

Importer le certificat client ***user.p12*** dans votre navigateur. :

Mot de passe : ***plb***

CA : ***PLB CA***

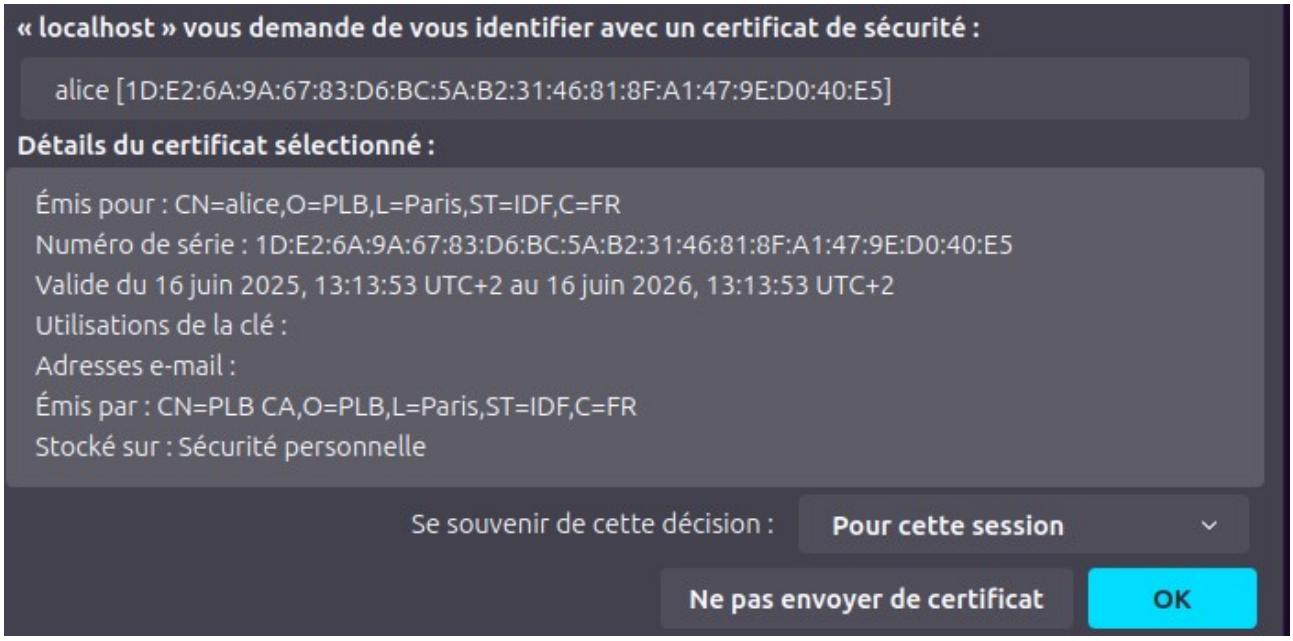
Par exemple, dans firefox :

Paramètres → Certificats → Afficher les certificats → Vos certificats → Importer

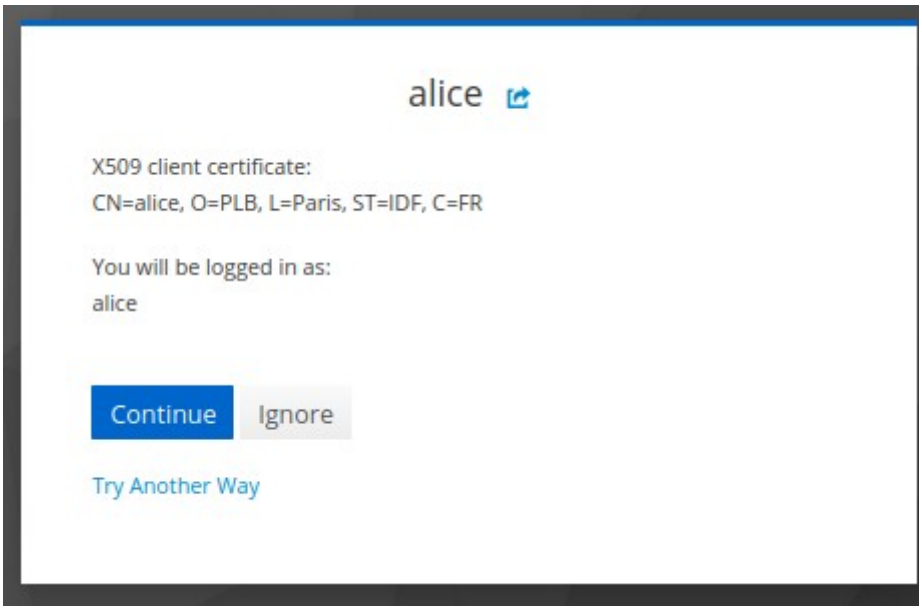
Accéder ensuite à :

https://localhost:8443/realms/formation/account

Vous devez voir la fenêtre suivante :



Envoyer le certificat et activer le bouton Sign in, vous devez voir la fenêtre suivante :



Et vous logger directement

A2 CIBA Flow (Optionnel)

A2.1 Mise en place realm

Visualiser le docker-compose fourni, en particulier la commande de démarrage.

Démarrer le docker compose :

```
docker compose up -d
```

Accéder à la console d'administration et :

- Créer un relm ***formation***
- Créer un client confidentiel ***ciba-client*** en activant le flow CIBA, noter son crédentiel
- Créer un utilisateur ***user*** sans mot de passe

A2.2 Application backend

Démarrer l'application backend :

```
cd ciba-backend
./mvnw clean package
java -jar target/ciba-backend-0.0.1-SNAPSHOT.jar
```

Le service démarre sur le port 8082

A2.3 Test du flow

Effectuer la requête de demande d'authentification auprès de Keycloak :

```
curl -u 'ciba-client:<client-secret>' \
  -d "scope=openid" \
  -d "login_hint=user" \
  -d "binding_message=TestCIBA" \
  http://localhost:8080/realms/formation/protocol/openid-connect/ext/ciba/auth
```

Noter le jeton de la réponse et Visualiser les logs de Keycloak et du backend.

Effectuer la requête de notification du résultat :

```
curl -X POST "http://localhost:8082/confirm?login_hint=user&status=SUCCEED"
```

Visualiser les logs du backend.

Finalement, effectuer la requête récupérant le jeton :

```
curl -X POST
http://localhost:8080/realms/formation/protocol/openid-connect/token \
  -u 'ciba-client:<client-secret>' \
```

```
  -d 'grant_type=urn:openid:params:grant-type:ciba' \
  -d 'auth_req_id=<token>'
```

A3 DPoP – Validation d'un appel API protégé

Objectif : Implémenter une protection DPoP sur une API et tester le rejet sans preuve

- Générer une clé (en JS)
- Obtenir un token depuis Keycloak avec DPoP activé
- Appeler une API sécurisée avec et sans en-tête DPoP
- Observer le rejet si le JWT est absent, ou que la clé ne correspond pas

Démarrer le docker compose du TP

- Importer le client ***dpop-client*** avec le json fourni ***dpop-client.json***

Vérifier la configuration dpop du client :
Advanced → Advanced settings

Visualiser le code su serveur d'API et le démarrer :

```
cd api-server
node api-server.js
```

Visualiser le code Javascript du client

Le code :

1. Génère une paire de clé
2. Calcule le thumbprint
3. Créé un DpoP pour obtenir un jeton
4. Obtiens un jeton d'accès en utilisant *client_credentials* et une entête DpOP
5. Recrée un DpOP pour accéder à l'api
6. Accède le service avec une requête DpOP
7. Accède le service avec un bearer simple

Exécuter et visualiser la console :

```
cd client
node dpop-client-credentials.js
```

A4 Usage du claim acr

Dans Realm settings, définir le mapping ACR to LoA suivant :

urn:formation:pwd = 1

urn:formation:mfa = 2

Créer un flow browser-loa comme suit :

Cookie ALTERNATIVE

Identity Provider Redirector ALTERNATIVEFormulaires (sous-flux) ALTERNATIVE

Niveau 1 (sous-flux) CONDITIONAL

Condition - Level of Authentication (LoA = 1)

Username Password Form REQUIRED

Niveau 2 (sous-flux) CONDITIONAL

Condition - Level of Authentication (LoA = 2)

OTP Form ALTERNATIVE

WebAuthn Authenticator ALTERNATIVE

Associer ce flow au client de l'application JavaScript fournie (Authentication flow overrides).

Avec l'application, envoyer acr_values=urn:formation:pwd puis acr_values=urn:formation:mfa dans la requête OIDC : comparer les étapes demandées et le claim acr de l'ID Token.

A4 Preuve de possession avec mTLS

Reprendre le client ***svc-client*** qui utilisait mTLS pour s'authentifier.

Lier le certificat au token :

Advanced → Advanced settings

Visualisez le code du serveur d'api fourni, il configure node.js en https et contient du code pour :

- Extrait le thumbprint du jeton
- Calcule le thumbprint attendu à partir du certificat
- Vérifie qu'il corresponde

Démarrer le serveur

```
npm install
node api-server.js
```

Obtenir un jeton auprès de Keycloak :

```
curl  -k   --cert user.crt   --key user.key  -X POST \
https://localhost:8443/realms/formation/protocol/openid-connect/token   \
-d grant_type=client_credentials   -d client_id=svc-client
```

Positionner TOKEN en variable d'environnement :

```
export TOKEN=
```

Accéder au serveur d'API :

```
curl -v --cert user.crt --key user.key   --cacert ca.crt \
  -H "Authorization: Bearer $TOKEN" \
 https://localhost:9443/resource
```

Vérifier la sortie du serveur d'API

A5 : Client policies

Dans ***Realm Settings – Client policies***

Définir un client profile ***pkce*** qui comporte l'exécuteur ***pkce-enforcer***

Définir une client policy ***pkce_4_all*** qui applique ce profil à tous les clients

Tester avec l'application frontend de l'atelier *3_oauth2* puis l'application frontend de l'atelier *5_Integration*